

网络安全技术

实 验 报 告

学 院	密网学院
年 级	2023 级
班 级	信息安全班
学 号	2310411
姓 名	李听泉
手机号	13803905146

2026 年 5 月 16 日

目录

1 实验目标	3
1.1 实验目的	3
1.2 实验要求与完成情况对照表	4
2 实验内容	6
2.1 RSA 公钥密码体制	6
2.2 RSA 密钥生成与加解密原理	8
2.3 DES 会话密钥分配机制	11
2.4 RSA-DES 混合加密通信原理	13
2.5 TCP 全双工通信与 select 机制	15
3 实验步骤	17
3.1 系统需求分析与总体设计	17
3.1.1 第一次实验项目基础	18
3.1.2 本次扩展需求	20
3.1.3 系统总体架构	22
3.1.4 通信协议设计	23
3.1.5 模块划分	26
3.2 RSA 算法模块设计与实现	29
3.2.1 设计思路	29
3.2.2 RSA 密钥生成实现	30
3.2.3 快速模幂与模逆元实现	32
3.2.4 DES 密钥的 RSA 分组加解密	34
3.3 DES 会话密钥自动生成	36
3.3.1 设计思路	36
3.3.2 自动生成函数实现	37
3.3.3 客户端生成与校验流程	38
3.4 RSA-DES 密钥交换流程设计	39
3.4.1 设计思路	39
3.4.2 密钥交换数据包格式	39
3.4.3 公钥包发送与接收实现	40
3.4.4 加密 DES 密钥包发送与接收实现	42
3.4.5 服务端与客户端密钥交换流程	45
3.5 DES 加密聊天功能复用	47
3.5.1 设计思路	47

3.5.2	实验二主控流程对原聊天功能的复用	48
3.5.3	DES 加密和解密函数复用	50
3.5.4	应用层封包机制复用	52
3.6	基于 select 的全双工通信实现	54
3.6.1	设计思路	54
3.6.2	select 全双工通信关键实现	54
3.6.3	实现效果分析	57
4	程序运行结果与测试分析	58
4.1	编译与运行环境	58
4.2	RSA 单元测试	60
4.3	自动密钥交换测试	62
4.4	服务端运行截图	64
4.5	客户端运行截图	66
4.6	全双工聊天测试	68
4.7	quit 退出测试	71
4.8	fork 与 select 模型对比	72
5	扩展与提高	74
5.1	RSA 分发 DES 会话密钥的安全性分析	75
5.2	混合加密方案的性能与适用性分析	76
5.3	实验方案的安全局限与改进方向	78
6	实验遇到的问题及其解决方法	81
6.1	问题一：RSA 不能直接加密完整 8 字节 DES 密钥	82
6.2	问题二：RSA 公钥和加密密钥包不能直接发送结构体	83
6.3	问题三：从 fork 模型扩展到 select 模型时需要同时处理键盘输入和 socket 接收	85
7	实验总结与个人思考	88

一、实验目标

1.1 实验目的

本实验在第一次实验“基于 DES 加密的 TCP 聊天程序”的基础上继续扩展，围绕 RSA 公钥密码体制、DES 会话密钥分配以及 Linux 平台下的 TCP 全双工通信机制，设计并完成一个“基于 RSA 算法自动分配 DES 密钥的加密聊天程序”。第一次实验中，通信双方需要预先约定并手工输入 DES 密钥；而在实际网络通信场景中，如何在不安全信道中安全地协商对称加密密钥，是保密通信系统设计中的核心问题之一。因此，本实验的主要目标是通过 RSA 非对称加密算法解决 DES 密钥分配问题，并在密钥协商完成后继续使用 DES 算法完成高效的加密聊天通信。

本实验的具体目的如下：

1. 加深对 RSA 公钥密码体制基本思想的理解，掌握公钥、私钥、模数、欧拉函数、模逆元和模幂运算在 RSA 算法中的作用。
2. 掌握 RSA 密钥生成、加密和解密的基本过程，理解 RSA 算法安全性与大整数分解困难问题之间的关系。
3. 理解对称加密算法在密钥分配方面存在的困难，掌握利用 RSA 分发 DES 会话密钥的混合加密通信思想。
4. 在第一次实验已有 DES 加密聊天程序的基础上，进一步实现 DES 会话密钥的自动生成与自动分发，使加密过程对用户更加透明。
5. 掌握 Linux Socket 编程中 TCP 全双工通信的实现方法，并进一步理解 `select()` 多路复用机制在网络通信程序中的作用。
6. 通过单元测试、集成测试和通信运行测试，验证 RSA 密钥分发、DES 加密通信以及全双工聊天流程的正确性和可靠性。

¹本项目所有源码均已上传至个人 github 仓库：

<https://github.com/Spring-LT/NetworkSecurity-lab2>

总体而言，本实验并不是简单地将 RSA 算法孤立实现，而是将 RSA 非对称加密、DES 对称加密和 TCP 网络通信结合起来，构造一个具备完整密钥协商流程和加密通信流程的实验型安全聊天系统。实验总体目标如图 1 所示。

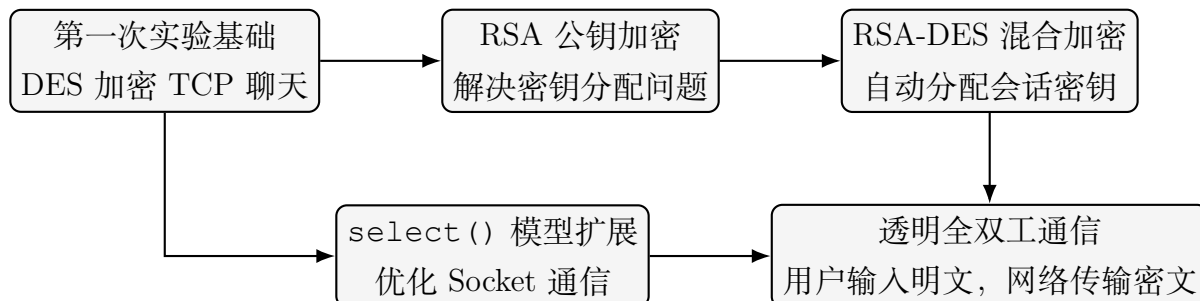


图 1: 本实验总体目标示意图

1.2 实验要求与完成情况对照表

根据实验任务要求，本次实验需要在第一次实验“基于 DES 加密的 TCP 通信”的基础上进行二次开发，使原有程序具备自动生成 DES 密钥、基于 RSA 算法完成密钥分配以及继续进行 DES 加密聊天的能力。同时，实验要求程序能够实现全双工通信，并保证加密和解密过程对用户透明。对于有能力的同学，还可以进一步使用 `select` 模型或异步 I/O 模型优化 Socket 通信部分。

本实验要求与当前完成情况的对应关系如表 1 所示。

表 1: 实验要求与完成情况对照表

序号	实验要求	具体说明	完成情况
1	Linux 平台下完成程序编写	程序应在 Linux 操作系统中基于 Socket 编程实现，能够完成服务端监听、客户端连接和双向通信。	已完成
2	保留第一次实验全部功能	在第一次实验 DES 加密 TCP 聊天程序基础上扩展，保留 DES 加密、TCP 通信、完整收发和正常退出等功能。	已完成
3	DES 密钥自动生成	通信双方不再依赖用户手工输入 DES 密钥，而是由程序自动生成会话密钥。	已完成
4	RSA 密钥分配	服务端生成 RSA 公私钥对，向客户端发送公钥；客户端使用 RSA 公钥加密 DES 会话密钥并发送给服务端。	已完成
5	DES 加密通信	RSA 仅用于分发 DES 会话密钥；正式聊天内容仍使用 DES 算法进行加密和解密。	已完成
6	全双工通信	通信双方均可以发送和接收消息，聊天过程不要求固定一问一答顺序。	已完成
7	加密过程对用户透明	用户在终端输入和看到的均为明文；程序内部自动完成发送前加密和接收后解密。	已完成
8	扩展要求：优化 Socket 通信模型	在保留原有 fork() 全双工模型基础上，进一步实现基于 select() 的单进程 I/O 多路复用通信模型。	已完成
9	测试与验证	通过 RSA 单元测试、自动密钥交换测试、聊天通信测试和退出测试验证程序正确性。	已完成

本实验不仅完成了 RSA 自动分配 DES 密钥的基础要求,还进一步完成了 `select()` 模型下的全双工通信扩展,使程序在功能完整性、工程结构和实验验证方面均较为充分。

二、实验内容

2.1 RSA 公钥密码体制

传统的对称密码体制要求通信双方使用同一把密钥完成加密和解密。其基本形式可以表示为:

$$C = E_K(M), \quad M = D_K(C) \quad (1)$$

其中, M 表示明文, C 表示密文, K 表示通信双方共享的密钥, $E_K(\cdot)$ 表示使用密钥 K 的加密算法, $D_K(\cdot)$ 表示使用同一密钥 K 的解密算法。

对称密码体制的优点是运算速度快,适合加密大量数据;但其核心困难在于密钥分配。如果通信双方在建立安全信道之前尚未共享密钥,那么如何在不安全网络环境中传递该密钥就成为系统安全性的关键问题。一旦密钥在传输过程中被窃听者获得,后续全部通信内容都可能被解密。

公钥密码体制,也称非对称密码体制,通过引入一对不同但数学相关的密钥来解决这一问题。每个用户拥有一对密钥:

- 公钥 K_{pub} : 可以公开给其他通信方,用于加密数据或验证签名。
- 私钥 K_{pri} : 只能由密钥拥有者秘密保存,用于解密数据或生成签名。

在公钥加密场景中,发送方使用接收方的公钥加密消息,接收方再使用自己的私钥解密消息,其基本过程为:

$$C = E_{K_{pub}}(M) \quad (2)$$

$$M = D_{K_{pri}}(C) \quad (3)$$

公钥密码体制要求算法满足如下性质：

1. 已知公钥 K_{pub} 时，可以较容易地完成加密运算。
2. 只有掌握私钥 K_{pri} 的接收方可以较容易地完成解密运算。
3. 在不知道私钥的情况下，攻击者即使获得公钥和密文，也难以恢复明文。
4. 从公钥推导出私钥在计算上应当不可行。

RSA 是最经典的公钥密码算法之一，其安全性建立在大整数因式分解困难问题之上。也就是说，对于两个大素数 p 和 q ，计算它们的乘积

$$n = pq \quad (4)$$

是容易的；但是仅根据 n 反推出 p 和 q 在计算上非常困难。当 p 和 q 足够大时，攻击者难以通过分解 n 获得私钥相关参数，从而保证 RSA 加密体系的安全性。

RSA 公钥密码体制的基本通信模型如图 2 所示。

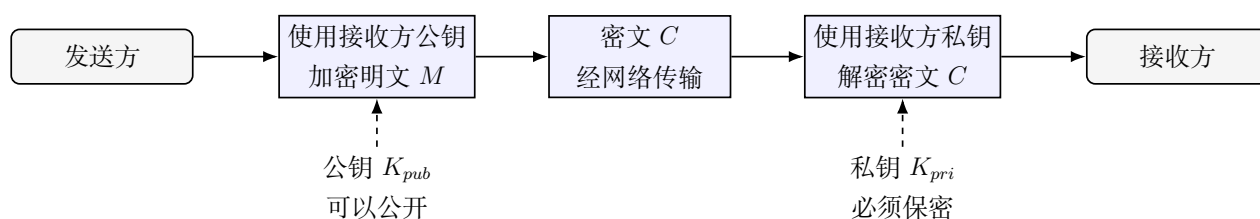


图 2: RSA 公钥密码体制基本通信模型

从图 2 可以看出，RSA 与 DES 等对称加密算法的最大区别在于：加密密钥和解密密钥不同。公钥可以通过网络公开发送，而私钥无需离开本地主机。这一特点使 RSA 非常适合用于解决对称密钥的安全分发问题。

2.2 RSA 密钥生成与加解密原理

RSA 算法的核心包括密钥生成、加密和解密三个过程。其数学基础主要包括素数、欧拉函数、模运算、模逆元和快速模幂运算。

1. RSA 密钥生成原理

RSA 密钥生成过程如下。

首先，随机选取两个不同的大素数 p 和 q ，计算模数 n ：

$$n = pq \quad (5)$$

其中， n 将同时出现在公钥和私钥中。接着计算欧拉函数 $\varphi(n)$ 。由于 p 和 q 为素数，因此有：

$$\varphi(n) = \varphi(pq) = (p-1)(q-1) \quad (6)$$

然后选择一个整数 e ，使其满足：

$$1 < e < \varphi(n), \quad \gcd(e, \varphi(n)) = 1 \quad (7)$$

其中， e 被称为公开指数。因为 e 与 $\varphi(n)$ 互素，所以 e 关于模 $\varphi(n)$ 存在乘法逆元。也就是说，可以找到一个整数 d ，使得：

$$ed \equiv 1 \pmod{\varphi(n)} \quad (8)$$

等价地，可以写为：

$$ed = 1 + k\varphi(n), \quad k \in \mathbb{Z} \quad (9)$$

其中， d 被称为私有指数。最终，RSA 的公钥和私钥分别为：

$$K_{pub} = (e, n) \quad (10)$$

$$K_{pri} = (d, n) \quad (11)$$

其中, (e, n) 可以公开, 而 (d, n) 必须由接收方秘密保存。RSA 密钥生成过程如图 3 所示。

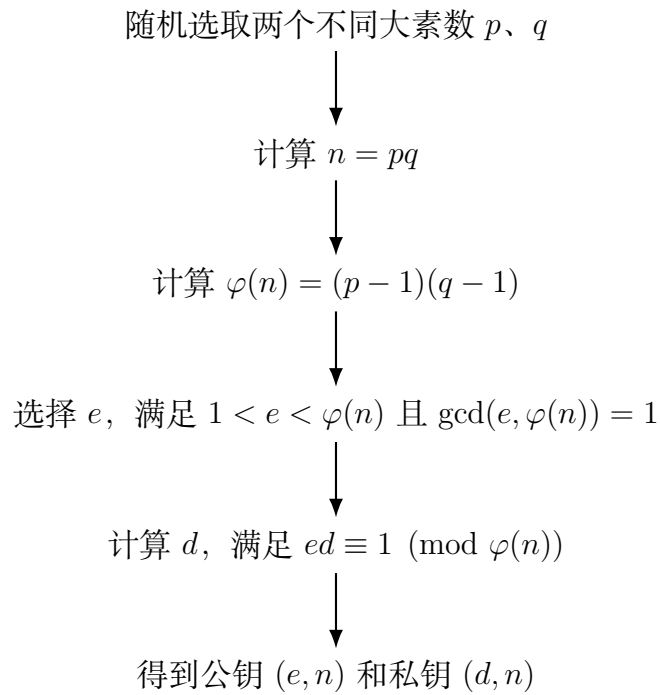


图 3: RSA 密钥生成流程

2. RSA 加密原理

设明文分组为 M , 且满足:

$$0 \leq M < n \quad (12)$$

发送方使用接收方公钥 (e, n) 对明文 M 进行加密, 得到密文 C :

$$C \equiv M^e \pmod{n} \quad (13)$$

该过程也可以写为：

$$C = E_{(e,n)}(M) \quad (14)$$

由于 e 和 n 是公开的，因此任何人都可以使用接收方的公钥加密数据。但是，只有掌握私钥 d 的接收方能够高效地从密文中恢复出明文。

3. RSA 解密原理

接收方收到密文 C 后，使用私钥 (d, n) 解密：

$$M \equiv C^d \pmod{n} \quad (15)$$

也可以写为：

$$M = D_{(d,n)}(C) \quad (16)$$

将加密公式代入解密公式可得：

$$C^d \equiv (M^e)^d \equiv M^{ed} \pmod{n} \quad (17)$$

由于密钥生成时满足：

$$ed \equiv 1 \pmod{\varphi(n)} \quad (18)$$

因此存在整数 k ，使得：

$$ed = 1 + k\varphi(n) \quad (19)$$

若 M 与 n 互素，根据欧拉定理有：

$$M^{\varphi(n)} \equiv 1 \pmod{n} \quad (20)$$

于是：

$$M^{ed} = M^{1+k\varphi(n)} = M \left(M^{\varphi(n)} \right)^k \equiv M \pmod{n} \quad (21)$$

这说明 RSA 解密能够正确恢复原始明文。对于一般情形，也可以通过中国剩余定理从模 p 和模 q 的角度证明 RSA 解密的正确性。

4. RSA 算法特点

RSA 算法具有如下特点：

1. 公钥可以公开发布，适合在不安全信道中完成密钥协商。
2. 私钥无需传输，只需由接收方本地保存。
3. 加密和解密涉及大整数模幂运算，计算成本高于对称加密算法。
4. RSA 适合加密较短数据，例如会话密钥、摘要值等，不适合直接加密大量聊天正文。
5. RSA 安全性依赖于模数 n 的长度以及攻击者分解 n 的难度。

因此，在实际安全通信系统中，RSA 往往不直接承担大量数据的加密任务，而是用于安全分发对称加密算法所需的会话密钥。

2.3 DES 会话密钥分配机制

DES 是一种典型的对称分组加密算法，其基本特点是加密和解密使用同一把密钥。设通信双方共享的 DES 密钥为 K_{DES} ，则明文 M 的加密和解密过程可以表示为：

$$C = E_{DES}(M, K_{DES}) \quad (22)$$

$$M = D_{DES}(C, K_{DES}) \quad (23)$$

其中, E_{DES} 表示 DES 加密函数, D_{DES} 表示 DES 解密函数。DES 的分组长度为 64 bit, 密钥输入长度通常为 64 bit, 其中 56 bit 为有效密钥位, 其余位可作为校验位使用。在本实验中, 使用 8 字节字符串表示 DES 密钥。

对称加密算法的优点是运算效率较高, 适合处理连续、大量的通信数据。但是, 对称加密算法要求通信双方在加密通信开始之前已经共享相同的密钥。若通信双方通过明文形式传递 DES 密钥, 则攻击者只要监听网络即可获得该密钥, 进而解密后续通信内容。因此, 对称密钥分配问题是 DES 加密通信系统必须解决的问题。

在网络通信中, 可以将每次连接过程中临时使用的对称密钥称为会话密钥。会话密钥通常只在一次通信会话中使用, 通信结束后即失效。其基本思想是:

1. 每次建立连接时生成新的随机密钥。
2. 使用该密钥加密当前会话中的通信数据。
3. 会话结束后丢弃该密钥。
4. 下次通信重新生成新的会话密钥。

会话密钥机制可以降低长期密钥泄露带来的风险。即使某一次会话密钥被攻击者获得, 也只影响当前会话, 而不会直接影响其他会话。

DES 会话密钥分配问题可以抽象为图 4 所示。

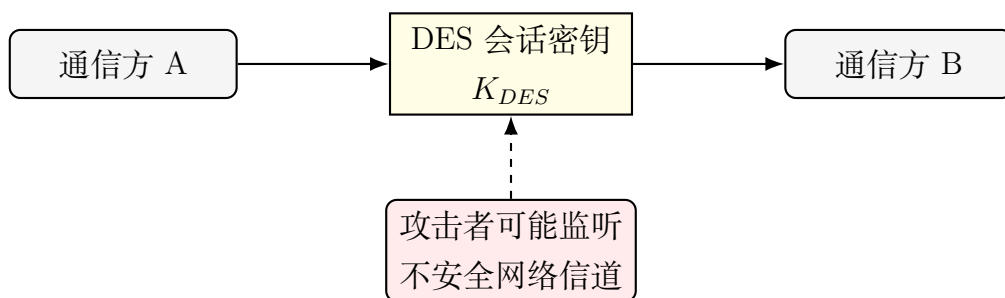


图 4: DES 会话密钥分配问题示意图

从图 4 可以看出, DES 加密通信的安全性不仅依赖 DES 算法本身, 还依赖于密钥分配过程是否安全。如果密钥分配过程不安全, 则后续即使使用 DES 加密传输, 通信内容仍可能被攻击者解密。因此, 需要引入一种安全的密钥分发机制, 使通信双方能够在不安全信道中共享 DES 会话密钥。

2.4 RSA-DES 混合加密通信原理

RSA-DES 混合加密通信结合了非对称加密和对称加密的优点。其基本思想是：

- 使用 RSA 解决 DES 会话密钥的安全分发问题。
- 使用 DES 完成后续聊天数据的高效加密传输。

在该混合加密方案中，RSA 不直接加密大量聊天正文，而只加密较短的 DES 会话密钥。假设客户端生成的 DES 会话密钥为 K_{DES} ，服务端 RSA 公钥为 (e, n) ，私钥为 (d, n) ，则密钥分发过程可以表示为：

$$C_K \equiv K_{DES}^e \pmod{n} \quad (24)$$

其中， C_K 表示经过 RSA 公钥加密后的 DES 会话密钥。服务端收到 C_K 后，使用私钥解密：

$$K_{DES} \equiv C_K^d \pmod{n} \quad (25)$$

当双方都获得相同的 K_{DES} 后，后续聊天内容 M_i 使用 DES 加密传输：

$$C_i = E_{DES}(M_i, K_{DES}) \quad (26)$$

接收方再使用相同的会话密钥解密：

$$M_i = D_{DES}(C_i, K_{DES}) \quad (27)$$

因此，RSA-DES 混合加密通信可分为两个阶段：

1. 密钥协商阶段：利用 RSA 公钥加密 DES 会话密钥，实现安全密钥分发。
2. 数据通信阶段：利用 DES 会话密钥加密聊天正文，实现高效加密通信。

其理论通信流程如图 5 所示。

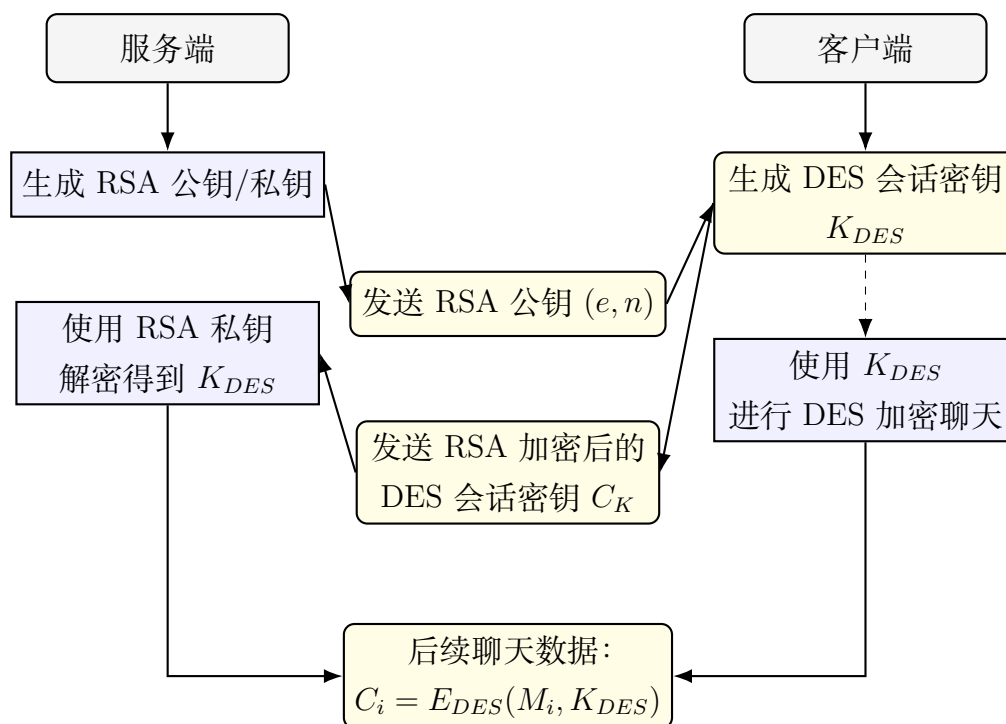


图 5: RSA-DES 混合加密通信原理

混合加密方案的优势在于能够同时利用两类密码体制的优点：

表 2: RSA 与 DES 在混合加密方案中的分工

算法	主要作用	优点	局限
RSA	加密并分发 DES 会话密钥	公钥可公开，适合在不安 全信道中完成密钥协商	模幂运算开销 较大，不适合加 密大量数据
DES	加密后续聊天正文	对称加密速度较快，适合 持续通信数据加密	通信前必须安 全共享密钥
RSA-DES 混合 方案	RSA 负责密钥分发， DES 负责数据加密	同时兼顾密钥分发安全性 和数据加密效率	安 全 性 仍 受 RSA 参数长度、 DES 密钥长度 和具体工作模 式影响

从表 15 可以看出，RSA-DES 混合加密并不是简单地叠加两种算法，而是根据不同算法的特点进行合理分工。RSA 解决如何安全共享 DES 密钥的问题，DES 解决如

何高效加密通信内容的问题。这种思想也是现代安全通信协议中常见的设计思路。

2.5 TCP 全双工通信与 select 机制

TCP 是一种面向连接、可靠传输、基于字节流的传输层协议。在基于 TCP 的聊天程序中，通信双方建立连接后，连接的两个方向都可以独立传输数据。也就是说，TCP 连接天然支持全双工通信。

设通信双方为 A 和 B，则在同一个 TCP 连接中可以同时存在两个方向的数据流：

$$A \longrightarrow B \quad (28)$$

$$B \longrightarrow A \quad (29)$$

这意味着 A 在向 B 发送数据的同时，也可以接收来自 B 的数据；B 也同理。全双工通信模型如图 6 所示。

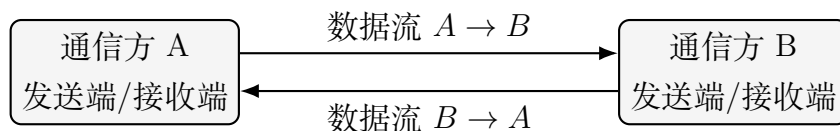


图 6: TCP 全双工通信模型

需要注意的是，TCP 面向应用层提供的是连续字节流，而不是天然带消息边界的数据报。因此，应用层不能简单假设“一次发送”必然对应“一次接收”。在实际通信中，可能出现粘包、拆包或接收不完整等现象。为了保证上层聊天消息能够被完整识别，通常需要在应用层设计封包机制，例如使用“长度头 + 数据体”的方式表示一条完整消息。

其抽象格式可以表示为：

$$\text{Packet} = \text{Length} \parallel \text{Payload} \quad (30)$$

其中，Length 表示后续数据体长度，Payload 表示实际传输的数据内容。对于加密

聊天程序而言，Payload 通常不是明文，而是加密后的密文数据。

1. 阻塞通信与全双工需求

在普通阻塞 I/O 模型下，如果程序直接调用阻塞式 `recv()` 等待网络数据，则在没有收到对端数据之前，该进程可能无法继续处理用户输入；反之，如果程序阻塞等待用户输入，也可能无法及时接收网络数据。因此，实现全双工聊天需要解决“同时关注键盘输入和网络 socket”的问题。

常见解决方式包括：

1. 使用多进程或多线程，将发送和接收分离到不同执行流中。
2. 使用 I/O 多路复用机制，在单进程中同时监听多个文件描述符。
3. 使用异步 I/O 或事件驱动框架处理输入输出事件。

其中，`select()` 是 Linux/Unix 系统中经典的 I/O 多路复用接口，适合用于课程实验中理解多路输入输出事件的处理方式。

2. select 机制基本原理

`select()` 的核心思想是：程序将一组需要关注的文件描述符交给内核，由内核阻塞等待这些描述符中的任意一个或多个变为“就绪”状态。当某些描述符可读、可写或发生异常时，`select()` 返回，程序再逐一检查哪些描述符已经就绪，并执行相应操作。

在聊天程序中，通常需要同时监听：

- 标准输入文件描述符 `STDIN_FILENO`：用于检测用户是否输入了新消息。
- 网络 socket 文件描述符：用于检测对端是否发送了新消息。

设监听集合为：

$$F = \{fd_{stdin}, fd_{socket}\} \quad (31)$$

则 `select()` 的作用可以抽象为：

$$F_{ready} = \{fd \mid fd \in F, fd \text{ 已就绪}\} \quad (32)$$

当 fd_{stdin} 就绪时，程序读取用户输入并发送消息；当 fd_{socket} 就绪时，程序接收网络数据并显示消息。这样即可在单进程中实现全双工聊天的效果。

`select()` 全双工通信原理如图 7 所示。

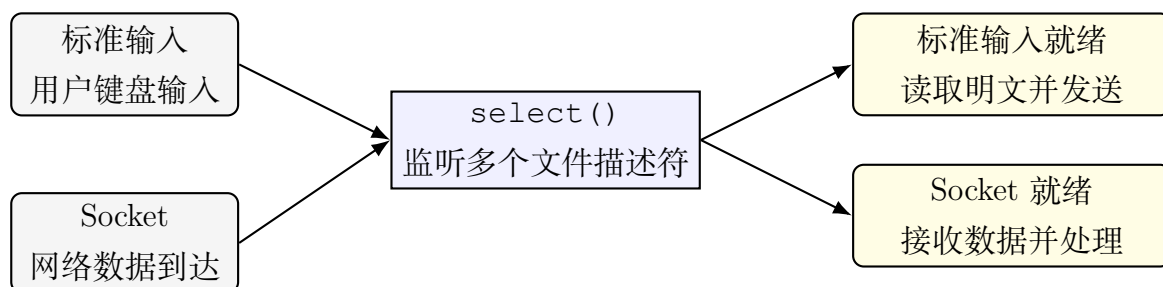


图 7: `select()` 多路复用实现全双工通信原理

相比简单阻塞式通信，`select()` 模型的优势在于：

1. 单进程即可同时关注键盘输入和网络 socket。
2. 避免单一阻塞点导致程序无法及时响应另一类事件。
3. 不需要创建额外进程或线程，程序结构更集中。
4. 适合一对一聊天程序、简单服务器和多连接实验场景。

因此，在基于 TCP 的加密聊天程序中，`select()` 可以用于优化通信模型，使程序在保持全双工通信能力的同时减少进程管理开销，并提高整体结构的清晰性。

三、实验步骤

3.1 系统需求分析与总体设计

本次实验是在第一次实验“基于 DES 加密的 TCP 聊天程序”基础上的二次扩展。第一次实验已经完成了 DES 算法实现、TCP 一对一通信、应用层封包、全双工聊天以

及异常退出处理等基础功能；本次实验的核心任务是在不破坏原有工程结构的前提下，引入 RSA 公钥密码体制，使程序能够自动生成 DES 会话密钥，并通过 RSA 算法完成密钥分配。密钥协商完成后，通信双方继续使用 DES 算法对聊天消息进行加密传输。

因此，本实验的总体设计思路可以概括为：保留第一次实验中已经验证正确的 DES 加密通信框架，新增 RSA 密钥生成与加解密模块、DES 会话密钥自动生成模块、RSA-DES 密钥交换模块，并进一步使用 `select()` 模型优化全双工通信过程。整个程序从功能上分为连接建立阶段、RSA-DES 密钥协商阶段和 DES 加密聊天阶段三个主要阶段。

3.1.1 第一次实验项目基础

第一次实验已经实现了一个完整的 Linux 平台 TCP 一对一加密聊天程序。该程序并不是简单的 Socket 通信示例，而是已经具备较完整的密码算法模块、网络通信模块和聊天控制模块，为本次实验扩展 RSA 自动密钥分配功能提供了良好的工程基础。

第一次实验项目的主要基础如表 3 所示。

表 3: 第一次实验项目基础

基础模块	已有实现内容	对本次实验的作用
DES 加解密模块	已实现 DES 初始置换、逆初始置换、E 扩展、P 置换、S 盒代换、PC-1、PC-2、16 轮 Feistel 结构、子密钥生成以及分组加解密流程。	本次实验继续使用 DES 作为会话数据加密算法，RSA 只负责分发 DES 会话密钥。
TCP 通信模块	已实现服务端 Socket 创建、端口绑定、监听、客户端连接、服务端接收连接等基础函数。	本次实验继续复用原有 TCP 建连流程，保证 RSA 密钥交换与 DES 聊天均运行在同一 TCP 连接之上。
完整收发机制	已实现 <code>sendAll()</code> 、 <code>recvAll()</code> 、 <code>sendPacket()</code> 、 <code>recvPacket()</code> 等函数，通过“长度头 + 数据体”解决 TCP 粘包、拆包问题。	本次实验中 RSA 公钥、RSA 加密后的 DES 会话密钥以及后续 DES 密文均可复用该封包机制传输。
聊天控制模块	已实现发送前加密、接收后解密、终端明文显示以及 <code>quit</code> 正常退出机制。	本次实验在完成 RSA-DES 密钥协商后，继续复用原有聊天流程，只需将手动输入的 DES 密钥替换为自动协商得到的会话密钥。
算法扩展与测试基础	原项目已扩展 3DES、AES-128，并包含单元测试和性能测试框架。	说明原工程具有较好的模块化基础，本次新增 RSA 模块和密钥交换测试可以自然接入现有测试体系。

从表 3 可以看出，第一次实验已经完成了通信程序中最基础、最容易出错的部分，包括 DES 加解密、TCP 完整收发、消息封包和全双工聊天。本次实验不需要重新设计整个聊天系统，而是在已有系统上增加自动密钥分发能力，使系统从“双方手工输入相同 DES 密钥”升级为“客户端自动生成 DES 密钥，服务端通过 RSA 私钥解密获得该密钥”。

第一次实验与本次实验之间的关系如图 8 所示。

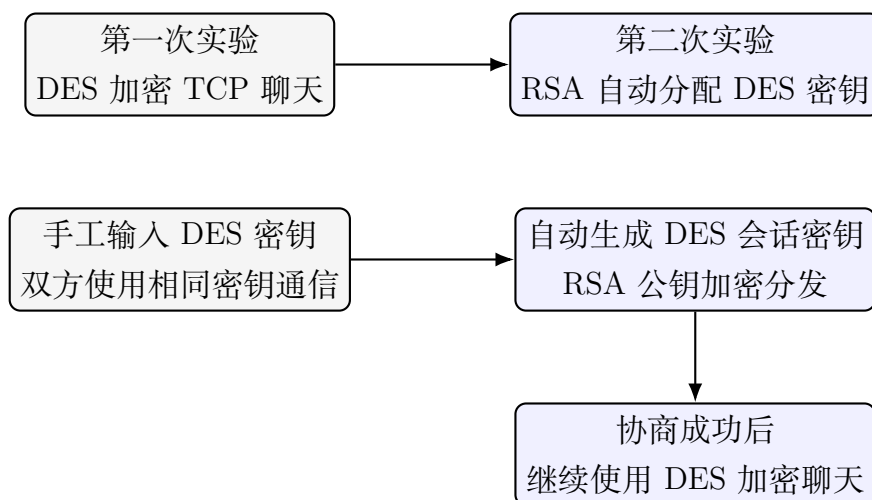


图 8: 第一次实验基础到本次实验扩展的关系

3.1.2 本次扩展需求

根据实验要求，本次扩展需要解决第一次实验中“DES 密钥需要通信双方预先手工输入”的问题。在实际网络环境中，如果直接通过明文方式传输 DES 密钥，则攻击者一旦监听到该密钥，后续所有 DES 加密消息均可能被解密。因此，本次实验需要引入 RSA 公钥密码算法，使通信双方能够在不安全信道上完成 DES 会话密钥的安全分配。

本次实验的功能需求可以分为基础功能需求、密码功能需求、通信功能需求和扩展提高需求四类，如表 4 所示。

表 4: 本次实验扩展需求分析

需求类别	需求内容	设计说明
基础功能需求	保留原有 DES 加密聊天功能	程序仍然需要完成 TCP 一对一聊天，用户输入明文，程序发送密文，接收端解密后显示明文。
密码功能需求	自动生成 DES 会话密钥	客户端在连接建立后自动生成 8 字节 DES 会话密钥，不再要求用户通过命令行参数或交互输入手动指定 DES 密钥。
密码功能需求	基于 RSA 分发 DES 密钥	服务端生成 RSA 公私钥对，并向客户端发送公钥；客户端使用服务端公钥加密 DES 会话密钥，服务端使用私钥解密得到该密钥。
通信功能需求	实现完整可靠的密钥交换流程	RSA 公钥、RSA 加密后的 DES 密钥以及后续 DES 密文均通过 TCP 连接传输，并继续使用应用层封包机制保证消息边界清晰。
通信功能需求	保证全双工聊天	密钥协商完成后，双方均可以随时发送和接收消息，不要求固定的请求-响应顺序。
用户体验需求	加密过程对用户透明	用户只需要启动服务端和客户端并输入聊天内容，不需要关心 DES 密钥生成、RSA 加密、RSA 解密和 DES 加解密细节。
扩展提高需求	使用 <code>select()</code> 优化通信模型	在保留第一次实验 <code>fork()</code> 全双工模型的基础上，进一步实现单进程 <code>select()</code> 多路复用通信模型，减少进程管理开销。

从功能流程上看，本次扩展后的程序需要满足以下运行逻辑：

1. 服务端启动并监听指定端口，等待客户端连接。
2. 客户端连接服务端，双方建立 TCP 连接。
3. 服务端生成 RSA 公钥和私钥，并将公钥发送给客户端。

4. 客户端自动生成 DES 会话密钥。
5. 客户端使用服务端 RSA 公钥加密 DES 会话密钥。
6. 客户端将 RSA 加密后的 DES 会话密钥发送给服务端。
7. 服务端使用 RSA 私钥解密，得到与客户端一致的 DES 会话密钥。
8. 双方使用该 DES 会话密钥进入加密聊天阶段。
9. 任意一方输入 quit 后，程序执行退出流程并释放资源。

上述过程使 DES 密钥不再以明文形式出现在网络中，从而解决了第一次实验中对称密钥需要提前共享的问题。

3.1.3 系统总体架构

本实验采用分层式总体架构设计。系统底层为 TCP Socket 通信层，负责连接建立、完整发送、完整接收和连接关闭；中间层为密码算法层，包括 DES 模块和 RSA 模块；其上是密钥交换层，负责组织 RSA 公钥发送、DES 会话密钥生成、DES 密钥 RSA 加密发送和 RSA 私钥解密；最上层是聊天业务层，负责读取用户输入、调用 DES 加密发送消息、接收密文并解密显示。

系统总体架构如图 9 所示。

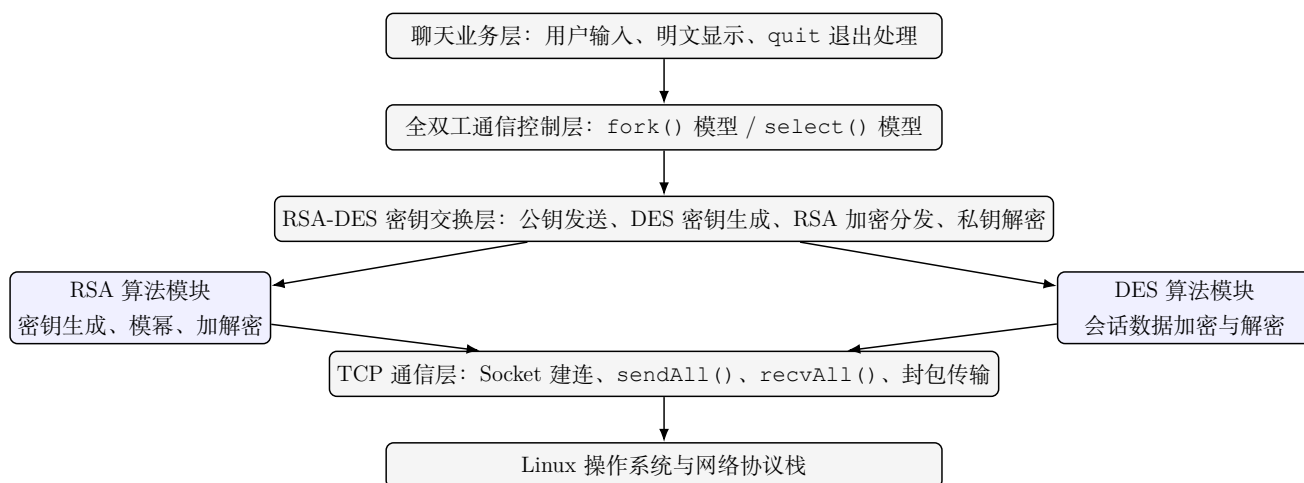


图 9: 系统总体架构设计

从通信双方角度看，服务端与客户端在功能上并不完全对称。服务端负责生成 RSA 密钥对，并保留 RSA 私钥；客户端负责生成 DES 会话密钥，并使用服务端公钥对其加密。密钥交换成功后，双方才进入对称的 DES 加密聊天阶段。

服务端与客户端的交互架构如图 10 所示。

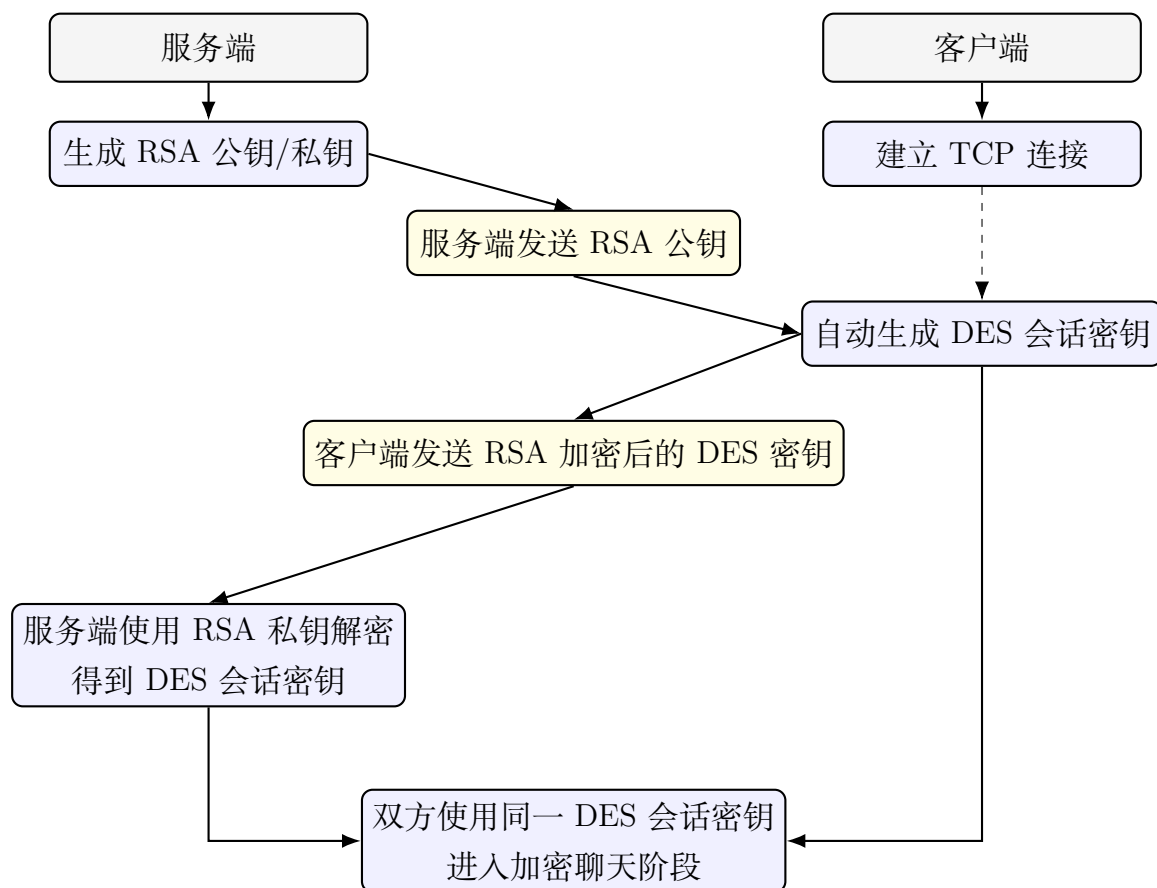


图 10: 服务端与客户端总体交互架构

需要特别说明的是，RSA 私钥始终只保存在服务端本地，不会通过网络发送。网络上传输的只有 RSA 公钥、经过 RSA 公钥加密后的 DES 会话密钥以及后续 DES 加密后的聊天密文。这样既符合公钥密码体制的基本原则，也避免了将私钥暴露给对端或网络监听者。

3.1.4 通信协议设计

由于 TCP 是面向字节流的协议，本身不保留应用层消息边界，因此本实验继续沿用第一次实验中已经实现的应用层封包机制。每一条应用层消息均由固定长度的长度头和可变长度的数据体组成。其基本格式如下：

$$\text{Packet} = \text{Length}_{4 \text{ bytes}} \parallel \text{Payload} \quad (33)$$

其中, $\text{Length}_{4 \text{ bytes}}$ 使用网络字节序表示后续数据体的字节长度, Payload 为具体数据内容。对于不同阶段, Payload 的含义不同: 在密钥交换阶段, 它可以是 RSA 公钥或 RSA 加密后的 DES 密钥; 在聊天阶段, 它是 DES 加密后的聊天密文。

本实验在原有“长度头 + 数据体”基础上进一步明确应用层数据体类型, 整体协议如表 5 所示。

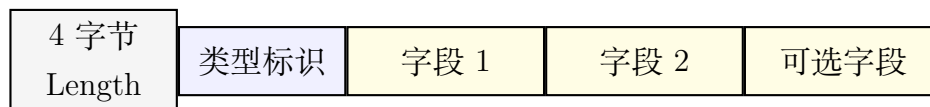
表 5: RSA-DES 聊天程序通信协议设计

阶段	数据包类型	数据体内容	说明
TCP 建连阶段	连接请求	TCP 三次握手由操作系统协议栈完成	应用程序通过服务端监听和客户端连接建立可靠字节流通道。
RSA 公钥发送阶段	RSA 公钥包	RSA 公钥参数 e, n	服务端生成 RSA 密钥对后, 仅将公钥发送给客户端, 私钥保留在服务端本地。
DES 密钥分发阶段	加密密钥包	RSA 加密后的 DES 会话密钥分组	客户端自动生成 DES 会话密钥, 并使用服务端 RSA 公钥加密后发送。
DES 聊天阶段	聊天密文包	DES 加密后的聊天消息密文	双方使用协商得到的同一 DES 会话密钥加密和解密聊天内容。
退出阶段	退出消息包	DES 加密后的 quit 或连接关闭信号	任意一方请求退出后, 程序释放 Socket 资源并结束聊天循环。

在具体数据编码上, RSA 公钥包和 RSA 加密后的 DES 密钥包需要能够在网络中准确还原整数参数, 因此本实验采用明确的字段组织方式, 而不是直接发送 C/C++ 结

构体内存。这样可以避免结构体填充字节、不同平台字节序以及字段对齐方式差异带来的兼容性问题。

本实验设计的密钥交换阶段逻辑包格式如图 11 所示。



RSA 公钥包：字段可表示 e, n

DES 密钥包：字段可表示 RSA 加密后的密钥分组

图 11: 密钥交换阶段逻辑包格式

对于聊天阶段，协议继续使用第一次实验中已经验证正确的“4 字节长度头 + 密文体”格式，如图 12 所示。



Length 表示密文长度，Payload 可以包含任意二进制字节，包括 $\backslash 0$

图 12: 聊天阶段应用层封包格式

通过上述协议设计，程序可以保证以下几点：

1. RSA 公钥、DES 会话密钥密文和聊天密文均通过统一的封包机制传输。
2. 每个应用层数据包都有明确长度，能够有效解决 TCP 粘包、拆包和半包问题。
3. 密钥交换阶段与聊天阶段在逻辑上清晰分离，便于程序调试和报告分析。
4. 网络中不会出现明文 DES 会话密钥，后续聊天内容也不会以明文形式传输。

本实验完整通信流程如图 13 所示。



图 13: RSA-DES 自动密钥分配聊天程序通信时序

3.1.5 模块划分

为了保持工程结构清晰，本实验采用模块化设计方式。原有 DES、TCP、聊天控制和工具函数模块继续保留；本次实验主要新增 RSA 算法模块、RSA-DES 密钥交换模块、实验二聊天主控模块、select() 全双工通信模块以及对应测试模块。

本次实验的模块划分如表 6 所示。

表 6: 系统模块划分

模块	主要文件	功能说明
RSA 算法模块	rsa.h、rsa.cpp	实现 RSA 密钥生成、最大公约数计算、扩展欧几里得求模逆元、快速模幂、素数生成、整数加密与解密等功能。

续表 6

模块	主要文件	功能说明
DES 加密模块	des.h、des.cpp	复用第一次实验中已经完成的 DES 算法实现，用于密钥协商完成后的聊天消息加密与解密。
RSA-DES 密钥交换模块	key_exchange.h、 key_exchange.cpp	负责组织服务端发送 RSA 公钥、客户端生成 DES 会话密钥、客户端 RSA 加密 DES 密钥、服务端 RSA 解密 DES 密钥等流程。
TCP 通信模块	tcp.h、tcp.cpp	复用原有 Socket 建连、完整发送、完整接收和应用层封装函数，为密钥交换和聊天数据传输提供可靠通道。
聊天控制模块	chat.h、chat.cpp	复用原有聊天逻辑，实现发送前加密、接收后解密、明文显示和 quit 退出处理。
select() 通信模块	chat_select.h、 chat_select.cpp	新增基于 select() 的全双工聊天模型，在单进程中同时监听标准输入和 Socket，提高通信模型的清晰性。

续表 6

模块	主要文件	功能说明
实验二主控模块	rsa_des_chat.h、 rsa_des_chat.cpp、 rsa_chat_main.cpp	提供本次实验专用服务端和客户端入口, 串联 TCP 建连、RSA-DES 密钥交换和 DES 加密聊天流程。
工具函数模块	utils.h、utils.cpp	提供参数解析、字节序转换、密钥检查、字节数组转换和十六进制输出等辅助功能。
测试模块	test_rsa.cpp、 test_key_exchange.cpp	对 RSA 算法正确性、DES 会话密钥自动生成以及 RSA-DES 密钥交换流程进行测试验证。

从模块依赖关系看, 本实验新增模块并没有破坏原有工程, 而是以“新增上层控制逻辑 + 复用底层稳定模块”的方式完成扩展。模块之间的依赖关系如图 14 所示。

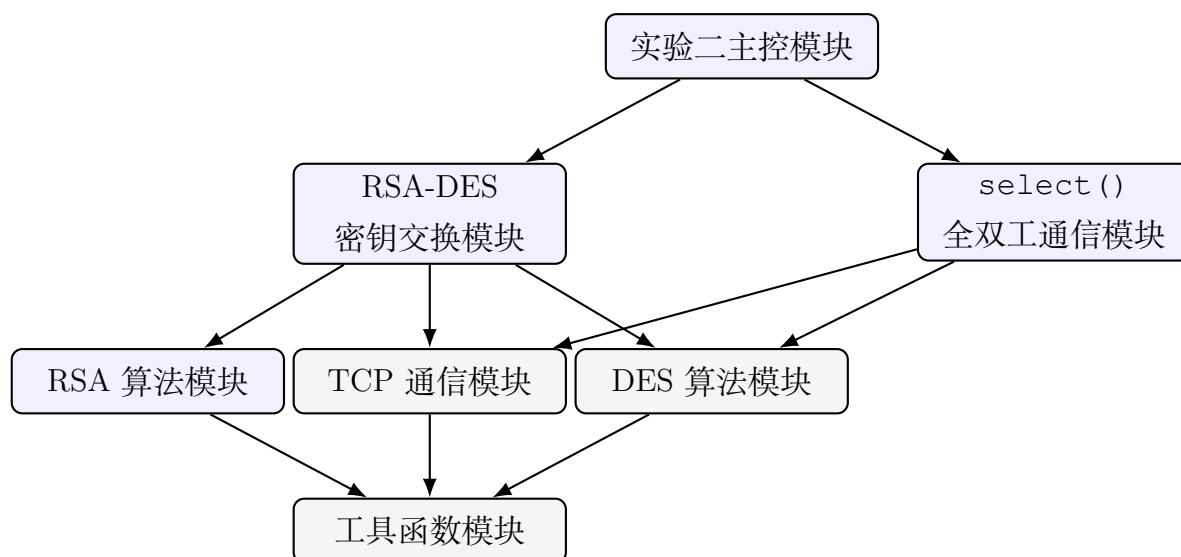


图 14: 系统模块依赖关系

通过上述模块划分，本实验实现了较清晰的职责分离：

1. RSA 模块只负责非对称密码算法本身，不直接处理 Socket。
2. DES 模块只负责对称加解密，不关心密钥如何产生和分发。
3. TCP 模块只负责可靠传输，不关心传输内容是公钥、密钥密文还是聊天密文。
4. 密钥交换模块负责将 RSA、DES 和 TCP 串联起来，完成实验二核心功能。
5. 聊天模块负责用户交互和全双工收发，使加密过程对用户透明。

这种模块化设计使程序具有较好的可维护性和可扩展性。后续如果需要将 DES 替换为 AES，或者将 RSA 参数长度进一步提高，只需要修改对应密码模块和密钥交换逻辑，而不需要重写 TCP 通信和聊天控制代码。

3.2 RSA 算法模块设计与实现

3.2.1 设计思路

RSA 算法模块是本次实验新增的核心密码算法模块，主要文件为 `rsa.h` 和 `rsa.cpp`。该模块不直接加密聊天正文，而是专门用于生成 RSA 公钥/私钥，并对客户端自动生成的 DES 会话密钥进行加密和解密。

本实验将 RSA 封装为 RSA 类，并定义 RsaPublicKey、RsaPrivateKey 和 RsaKeyPair 三个结构体。其中，公钥结构保存 e, n ，私钥结构保存 d, n ，完整密钥对结构额外保存 $p, q, \varphi(n)$ 等中间参数，便于调试和实验报告分析。服务端生成密钥对后，只发送公钥，私钥始终保存在服务端本地。

RSA 模块的核心任务包括：

1. 随机生成两个不同素数 p, q 。
2. 计算 $n = pq$ 和 $\varphi(n) = (p-1)(q-1)$ 。
3. 选择公开指数 e ，并保证 e 与 $\varphi(n)$ 互素。
4. 通过扩展欧几里得算法计算私钥指数 d ，满足 $ed \equiv 1 \pmod{\varphi(n)}$ 。
5. 将 8 字节 DES 密钥拆分为 4 个 16 bit 分组，并分别执行 RSA 加密和解密。

3.2.2 RSA 密钥生成实现

RSA 密钥生成过程由 `RSA::generateKeyPair()` 完成。程序首先调用 `randomPrime()` 随机生成两个不同素数 p, q ，然后计算 n 与 $\varphi(n)$ 。公开指数优先选用常见的 65537，如果不满足互素条件，则从较小奇数开始继续寻找合法的 e 。最后调用 `modInverse()` 求出私钥指数 d 。

```
1 bool RSA::generateKeyPair(int primeBits)
2 {
3     try
4     {
5         // 1. 随机生成两个不同素数 p、q。
6         std::uint64_t p = randomPrime(primeBits);
7         std::uint64_t q = randomPrime(primeBits);
8
9         while (q == p)
10        {
11            q = randomPrime(primeBits);
12        }
13
14        // 2. 计算 RSA 模数 n 和欧拉函数 phi(n)。
```

```
15     const std::uint64_t n = p * q;
16     const std::uint64_t phi = (p - 1) * (q - 1);
17
18     // 3. 优先选择常用公开指数 65537。
19     std::uint64_t e = 65537ULL;
20
21     if (e >= phi || gcd64(e, phi) != 1)
22     {
23         e = 3;
24         while (e < phi && gcd64(e, phi) != 1)
25         {
26             e += 2;
27         }
28     }
29
30     if (e >= phi)
31     {
32         return false;
33     }
34
35     // 4. 计算私钥指数 d, 使  $e*d \equiv 1 \pmod{\phi}$ 。
36     const std::uint64_t d = modInverse(e, phi);
37     if (d == 0)
38     {
39         return false;
40     }
41
42     // 5. 保存公钥、私钥和中间参数。
43     keyPair_.publicKey = RsaPublicKey{e, n};
44     keyPair_.privateKey = RsaPrivateKey{d, n};
45     keyPair_.p = p;
46     keyPair_.q = q;
47     keyPair_.phi = phi;
48     ready_ = true;
49
50     return true;
51 }
52 catch (...)
53 {
```



```

54     ready_ = false;
55     return false;
56 }
57 }

```

Listing 1 RSA 密钥对生成关键代码

上述代码中，`ready_` 用于标记当前 RSA 对象是否已经成功生成密钥。若密钥生成失败，后续获取公钥、私钥或进行解密操作时不会继续使用未初始化的密钥。

3.2.3 快速模幂与模逆元实现

RSA 加密和解密的核心运算为：

$$C \equiv M^e \pmod{n} \quad (34)$$

$$M \equiv C^d \pmod{n} \quad (35)$$

直接计算 M^e 或 C^d 会产生极大的中间数，因此程序使用二进制快速模幂算法。为了减少 64 位乘法中间结果溢出的风险，实际实现中还使用 `__uint128_t` 完成模乘。

```

1 std::uint64_t modPow(std::uint64_t base,
2                     std::uint64_t exp,
3                     std::uint64_t mod)
4 {
5     if (mod == 0)
6     {
7         throw std::invalid_argument("modPow(): mod must not be
8         zero");
9     }
10
11     // 1. 将底数先化到模空间内。
12     // 2. 使用二进制快速幂逐位处理指数。
13     // 3. 每次乘法使用 mulMod, 避免中间乘积溢出。
14     std::uint64_t result = 1 % mod;
15     base %= mod;

```

```
16     while (exp > 0)
17     {
18         if ((exp & 1ULL) != 0ULL)
19         {
20             result = mulMod(result, base, mod);
21         }
22
23         base = mulMod(base, base, mod);
24         exp >>= 1ULL;
25     }
26
27     return result;
28 }
```

Listing 2 快速模幂关键代码

私钥指数 d 通过扩展欧几里得算法求得。该方法比简单枚举更高效，也更符合 RSA 算法的数学原理。

```
1 std::uint64_t modInverse(std::uint64_t e, std::uint64_t phi)
2 {
3     if (phi == 0)
4     {
5         return 0;
6     }
7
8     // 1. 使用扩展欧几里得求  $e*x + phi*y = gcd(e, phi)$ 。
9     // 2. 若  $gcd$  不为 1，则说明逆元不存在。
10    // 3. 若  $x$  为负，将其调整到  $[0, phi)$  范围。
11    std::int64_t x = 0;
12    std::int64_t y = 0;
13
14    const std::int64_t g =
15        extendedGcd(static_cast<std::int64_t>(e),
16
17                    static_cast<std::int64_t>(phi),
18
19                    x,
20                    y);
```

```

19     if (g != 1)
20     {
21         return 0;
22     }
23
24     std::int64_t ans = x % static_cast<std::int64_t>(phi);
25     if (ans < 0)
26     {
27         ans += static_cast<std::int64_t>(phi);
28     }
29
30     return static_cast<std::uint64_t>(ans);
31 }

```

Listing 3 模逆元计算关键代码

3.2.4 DES 密钥的 RSA 分组加解密

本实验中 DES 会话密钥长度为 8 字节。由于 RSA 要求明文整数小于模数 n ，程序没有直接将 8 字节密钥整体作为一个大整数加密，而是将其拆成 4 个 16 bit 分组。每个分组分别执行 RSA 加密，服务端解密时再按相同顺序拼回 8 字节 DES 密钥。

$$K_{DES} = k_0 \parallel k_1 \parallel \cdots \parallel k_7 \quad (36)$$

$$M_0 = k_0 \parallel k_1, \quad M_1 = k_2 \parallel k_3, \quad M_2 = k_4 \parallel k_5, \quad M_3 = k_6 \parallel k_7 \quad (37)$$

```

1 std::vector<std::uint64_t> RSA::encryptDesKey(const std::string&
      desKey,
2
3                                     const
      RsaPublicKey& key)
4 {
5     if (desKey.size() != DES_KEY_SIZE)
6     {
7         throw std::invalid_argument("RSA::encryptDesKey(): DES
      key must be 8 bytes");
8     }
9 }

```

```
8
9 // 1. DES 密钥为 8 字节。
10 // 2. 每 2 字节合成一个 16 bit 整数。
11 // 3. 对 4 个整数分别执行 RSA 加密。
12 std::vector<std::uint64_t> cipherChunks;
13 cipherChunks.reserve(4);
14
15 for (std::size_t i = 0; i < desKey.size(); i += 2)
16 {
17     const std::uint64_t high =
18         static_cast<unsigned char>(desKey[i]);
19
20     const std::uint64_t low =
21         static_cast<unsigned char>(desKey[i + 1]);
22
23     const std::uint64_t plainChunk = (high << 8ULL) | low;
24     cipherChunks.push_back(encryptInteger(plainChunk, key));
25 }
26
27 return cipherChunks;
28 }
29
30 std::string RSA::decryptDesKey(const std::vector<std::uint64_t>&
31     cipherChunks) const
32 {
33     if (cipherChunks.size() != 4)
34     {
35         throw std::invalid_argument("RSA::decryptDesKey():
36             encrypted DES key must contain 4 chunks");
37     }
38
39     // 1. 分别解密 4 个 RSA 密文整数。
40     // 2. 每个整数拆回 2 字节。
41     // 3. 拼接得到 8 字节 DES 会话密钥。
42     std::string desKey;
43     desKey.reserve(DES_KEY_SIZE);
44
45     for (std::uint64_t cipherChunk : cipherChunks)
46     {
```

```
45     const std::uint64_t plainChunk =  
        decryptInteger(cipherChunk);  
46  
47     const char high =  
        static_cast<char>((plainChunk >> 8ULL) & 0xFFULL);  
48  
49  
50     const char low =  
        static_cast<char>(plainChunk & 0xFFULL);  
51  
52  
53     desKey.push_back(high);  
54     desKey.push_back(low);  
55 }  
56  
57 return desKey;  
58 }
```

Listing 4 DES 密钥的 RSA 加解密关键代码

通过上述设计，RSA 模块既完成了完整的公钥密码算法流程，又能够自然服务于 DES 会话密钥分发需求。

3.3 DES 会话密钥自动生成

3.3.1 设计思路

第一次实验中，DES 密钥由服务端和客户端分别手工输入。该方式虽然便于初步验证 DES 加密通信功能，但不符合本次实验自动分配密钥的要求，也无法模拟真实网络通信中会话密钥协商的过程。

本次实验将 DES 会话密钥的生成放在客户端完成。客户端在 TCP 连接建立后自动生成一个 8 字节随机字符串作为 DES 会话密钥，满足原 DES 模块中 `DES_KEY_SIZE = 8` 的要求。随后客户端使用服务端 RSA 公钥加密该密钥，并发送给服务端。服务端解密后即可得到同样的 DES 会话密钥。

为了便于实验环境下测试，并避免不可打印字符影响终端显示，本实验生成的 DES 会话密钥由数字、大写字母和小写字母组成。正式运行时程序不会直接打印密钥明文，而是以 `[hidden]` 形式提示密钥已生成。

3.3.2 自动生成函数实现

DES 会话密钥生成函数位于 `key_exchange.cpp` 中。其关键实现如下：

```
1 std::string generateRandomDesKey()
2 {
3     // 1. 为了实验截图可读性，随机密钥采用可打印字符。
4     // 2. 每次生成 8 字节，满足 DES 密钥长度要求。
5     // 3. 正式运行界面不直接打印密钥明文。
6     static const char charset[] =
7         "0123456789"
8         "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
9         "abcdefghijklmnopqrstuvwxyz";
10
11     static std::random_device rd;
12     static std::mt19937 gen(rd());
13
14     std::uniform_int_distribution<std::size_t> dist(0,
15         sizeof(charset) - 2);
16
17     std::string key;
18     key.reserve(DES_KEY_SIZE);
19
20     for (std::size_t i = 0; i < DES_KEY_SIZE; ++i)
21     {
22         key.push_back(charset[dist(gen)]);
23     }
24
25     return key;
26 }
```

Listing 5 DES 会话密钥自动生成关键代码

该函数使用 `std::random_device` 和 `std::mt19937` 生成随机下标，并从预设字符集中选取 8 个字符组成 DES 会话密钥。生成后，客户端会立即调用原有工具函数 `validateKeyForAlgorithm()` 对密钥合法性进行检查。

3.3.3 客户端生成与校验流程

客户端在 `clientPerformRsaDesKeyExchange()` 中完成 DES 会话密钥生成、校验、接收公钥和发送密钥密文。这里展示其中与自动生成密钥直接相关的关键逻辑：

```
1 std::string clientPerformRsaDesKeyExchange(int sockfd)
2 {
3     // 1. 客户端自动生成 8 字节 DES 会话密钥。
4     const std::string desKey = generateRandomDesKey();
5
6     // 2. 调用原有密钥校验函数，确保生成结果满足 DES 要求。
7     std::string reason;
8     if (!validateKeyForAlgorithm(desKey, AlgorithmType::DES,
9         &reason))
10    {
11        throw std::runtime_error("generated DES key is invalid:
12            " + reason);
13    }
14
15    // 3. 正式运行时不输出密钥明文，避免在终端泄露。
16    std::cout << "[DES] Random DES session key generated.
17        [hidden]\n";
18
19    // 4. 后续继续接收 RSA 公钥并发送加密后的 DES 密钥。
20    RsaPublicKey publicKey;
21    if (!recvRsaPublicKey(sockfd, publicKey))
22    {
23        throw std::runtime_error("receive RSA public key
24            failed");
25    }
26
27    if (!sendEncryptedDesKey(sockfd, desKey, publicKey))
28    {
29        throw std::runtime_error("send encrypted DES key
30            failed");
31    }
32
33    return desKey;
34 }
```

Listing 6 客户端生成并校验 DES 会话密钥

由此可见，客户端不再需要用户通过 `-k` 参数或交互输入方式提供 DES 密钥，而是在连接建立后自动生成密钥，并将其作为后续 DES 加密聊天的会话密钥。

3.4 RSA-DES 密钥交换流程设计

3.4.1 设计思路

RSA-DES 密钥交换模块位于 `key_exchange.h` 和 `key_exchange.cpp` 中，是本次实验连接 RSA 算法、DES 会话密钥和 TCP 通信的核心模块。该模块的基本思想是：RSA 只负责保护 DES 会话密钥，DES 负责后续聊天正文加密。

完整密钥交换流程如下：

1. 服务端生成 RSA 公钥和私钥。
2. 服务端通过 TCP 连接向客户端发送 RSA 公钥。
3. 客户端自动生成 8 字节 DES 会话密钥。
4. 客户端使用服务端 RSA 公钥加密 DES 会话密钥。
5. 客户端将加密后的 DES 会话密钥发送给服务端。
6. 服务端使用 RSA 私钥解密，恢复出 DES 会话密钥。
7. 双方使用同一个 DES 会话密钥进入后续加密聊天阶段。

3.4.2 密钥交换数据包格式

本实验没有直接发送 C++ 结构体内存，而是手动组织字节数组，避免结构体填充、字段对齐和主机字节序差异带来的问题。

RSA 公钥包格式如下：

$$\text{RPK1 Packet} = \text{Magic}_{4B} \parallel e_{8B} \parallel n_{8B} \quad (38)$$

其中, Magic 为 RPK1, 用于标识该数据包为 RSA 公钥包。

加密 DES 密钥包格式如下:

$$\text{EDK1 Packet} = \text{Magic}_{4B} \parallel \text{Count}_{4B} \parallel C_{08B} \parallel C_{18B} \parallel C_{28B} \parallel C_{38B} \quad (39)$$

其中, Magic 为 EDK1, Count 固定为 4, 表示 8 字节 DES 密钥被拆成 4 个 RSA 密文分组。

3.4.3 公钥包发送与接收实现

公钥包发送函数 `sendRsaPublicKey()` 将 RPK1、公开指数 e 和模数 n 依次写入字节数组,并复用第一次实验中的 `sendPacket()` 完整发送。接收端通过 `recvRsaPublicKey()` 接收完整包,检查 magic 和长度后解析出 e, n 。

```

1 bool sendRsaPublicKey(int sockfd, const RsaPublicKey& publicKey)
2 {
3     // 1. 检查公钥参数是否合法。
4     if (publicKey.e == 0 || publicKey.n == 0)
5     {
6         return false;
7     }
8
9     // 2. 组织公钥包: [4 bytes magic = RPK1][8 bytes e][8 bytes n]。
10    std::vector<unsigned char> packet;
11    packet.reserve(20);
12
13    appendMagic(packet, RSA_PUBLIC_KEY_MAGIC);
14    appendUint64(packet, publicKey.e);
15    appendUint64(packet, publicKey.n);
16
17    // 3. 复用原有 sendPacket() 完整发送。
18    return sendPacket(sockfd, packet);
19 }
20
21 bool recvRsaPublicKey(int sockfd, RsaPublicKey& publicKey)
22 {
23     std::vector<unsigned char> packet;

```

```
24
25 // 1. 接收完整公钥包。
26 if (!recvPacket(sockfd, packet))
27 {
28     return false;
29 }
30
31 // 2. 检查长度和 magic, 确保包类型正确。
32 if (packet.size() != 20 || !checkMagic(packet,
33     RSA_PUBLIC_KEY_MAGIC))
34 {
35     return false;
36 }
37
38 // 3. 按大端序解析 e 和 n。
39 std::size_t offset = 4;
40 std::uint64_t e = 0;
41 std::uint64_t n = 0;
42
43 if (!readUInt64(packet, offset, e))
44 {
45     return false;
46 }
47
48 if (!readUInt64(packet, offset, n))
49 {
50     return false;
51 }
52
53 // 4. 简单合法性检查后保存公钥。
54 if (e == 0 || n == 0)
55 {
56     return false;
57 }
58
59 publicKey.e = e;
60 publicKey.n = n;
61 return true;
62 }
```

Listing 7 RSA 公钥包发送与接收关键代码**3.4.4 加密 DES 密钥包发送与接收实现**

客户端调用 `sendEncryptedDesKey()` 对 DES 会话密钥进行 RSA 加密，并将得到的 4 个密文分组组织为 EDK1 数据包。服务端调用 `recvEncryptedDesKey()` 接收该数据包，并使用 RSA 私钥恢复 DES 会话密钥。

```
1 bool sendEncryptedDesKey(int sockfd,
2                           const std::string& desKey,
3                           const RsaPublicKey& publicKey)
4 {
5     // 1. 检查 DES 密钥长度是否为 8 字节。
6     if (desKey.size() != DES_KEY_SIZE)
7     {
8         return false;
9     }
10
11    // 2. 使用 RSA 公钥加密 DES 密钥，得到 4 个密文分组。
12    std::vector<std::uint64_t> cipherChunks;
13
14    try
15    {
16        cipherChunks = RSA::encryptDesKey(desKey, publicKey);
17    }
18    catch (...)
19    {
20        return false;
21    }
22
23    // 3. 组织加密密钥包:
24    // [4 bytes magic = EDK1][4 bytes chunkCount][8 bytes
25    //    C0]...[8 bytes C3]
26    std::vector<unsigned char> packet;
27    packet.reserve(4 + 4 + cipherChunks.size() * 8);
28
29    appendMagic(packet, ENCRYPTED_DES_KEY_MAGIC);
```

```
29     appendUint32(packet,  
30         static_cast<std::uint32_t>(cipherChunks.size()));  
31  
32     for (std::uint64_t cipher : cipherChunks)  
33     {  
34         appendUint64(packet, cipher);  
35     }  
36  
37     // 4. 复用原有 sendPacket() 完整发送。  
38     return sendPacket(sockfd, packet);  
39 }  
40  
41 bool recvEncryptedDesKey(int sockfd,  
42                          const RSA& rsa,  
43                          std::string& desKey)  
44 {  
45     std::vector<unsigned char> packet;  
46  
47     // 1. 接收完整加密密钥包。  
48     if (!recvPacket(sockfd, packet))  
49     {  
50         return false;  
51     }  
52  
53     // 2. 检查 magic 和基本长度。  
54     if (packet.size() < 8 || !checkMagic(packet,  
55         ENCRYPTED_DES_KEY_MAGIC))  
56     {  
57         return false;  
58     }  
59  
60     // 3. 读取密文分组数量，并确认数量为 4。  
61     std::size_t offset = 4;  
62     std::uint32_t count = 0;  
63  
64     if (!readUint32(packet, offset, count))  
65     {  
66         return false;  
67     }  
68 }
```

```
66
67     if (count != 4)
68     {
69         return false;
70     }
71
72     if (packet.size() != 4 + 4 + static_cast<std::size_t>(count)
73         * 8)
74     {
75         return false;
76     }
77
78     // 4. 读取 4 个 RSA 密文分组。
79     std::vector<std::uint64_t> cipherChunks;
80     cipherChunks.reserve(count);
81
82     for (std::uint32_t i = 0; i < count; ++i)
83     {
84
85         std::uint64_t cipher = 0;
86
87         if (!readUint64(packet, offset, cipher))
88         {
89             return false;
90         }
91
92         cipherChunks.push_back(cipher);
93
94         // 5. 使用服务端 RSA 私钥解密, 恢复 DES 会话密钥。
95         try
96         {
97             desKey = rsa.decryptDesKey(cipherChunks);
98         }
99         catch (...)
100         {
101             return false;
102         }
103
104     return desKey.size() == DES_KEY_SIZE;
```

104 }

Listing 8 RSA 加密 DES 密钥包发送与接收关键代码**3.4.5 服务端与客户端密钥交换流程**

服务端完整密钥交换由 `serverPerformRsaDesKeyExchange()` 负责。该函数生成 RSA 密钥对，发送公钥，接收加密后的 DES 密钥，并使用私钥解密恢复出 DES 会话密钥。

```
1 std::string serverPerformRsaDesKeyExchange(int sockfd)
2 {
3     // 1. 服务端生成 RSA 公私钥对。
4     RSA rsa;
5     if (!rsa.generateKeyPair(16))
6     {
7         throw std::runtime_error("RSA key pair generation
8             failed");
9     }
10    const RsaKeyPair keyPair = rsa.getKeyPair();
11
12    std::cout << "[RSA] Key pair generated.\n";
13    std::cout << "[RSA] Public key: e=" << keyPair.publicKey.e
14        << ", n=" << keyPair.publicKey.n << "\n";
15    std::cout << "[RSA] Private key is kept only on server.
16        [hidden]\n";
17
18    // 2. 服务端只发送公钥，不发送私钥。
19    if (!sendRsaPublicKey(sockfd, keyPair.publicKey))
20    {
21        throw std::runtime_error("send RSA public key failed");
22    }
23
24    // 3. 服务端接收 RSA 加密后的 DES 会话密钥。
25    std::string desKey;
26    if (!recvEncryptedDesKey(sockfd, rsa, desKey))
```

```

27     throw std::runtime_error("receive encrypted DES key
28     failed");
29 }
30 // 4. 服务端检查解密得到的 DES 会话密钥是否合法。
31 std::string reason;
32 if (!validateKeyForAlgorithm(desKey, AlgorithmType::DES,
33     &reason))
34 {
35     throw std::runtime_error("decrypted DES key is invalid:
36     " + reason);
37 }
38 std::cout << "[RSA] DES session key decrypted successfully.
39     [hidden]\n";
40 return desKey;
41 }

```

Listing 9 服务端 RSA-DES 密钥交换关键代码

客户端完整密钥交换由 `clientPerformRsaDesKeyExchange()` 负责。该函数自动生成 DES 会话密钥，接收服务端公钥，并使用该公钥加密 DES 密钥后发送。

```

1 std::string clientPerformRsaDesKeyExchange(int sockfd)
2 {
3     // 1. 客户端自动生成 8 字节 DES 会话密钥。
4     const std::string desKey = generateRandomDesKey();
5
6     std::string reason;
7     if (!validateKeyForAlgorithm(desKey, AlgorithmType::DES,
8         &reason))
9     {
10         throw std::runtime_error("generated DES key is invalid:
11         " + reason);
12     }
13
14     std::cout << "[DES] Random DES session key generated.
15     [hidden]\n";

```

```
14 // 2. 客户端接收服务端 RSA 公钥。
15 RsaPublicKey publicKey;
16
17 if (!recvRsaPublicKey(sockfd, publicKey))
18 {
19     throw std::runtime_error("receive RSA public key
20     failed");
21 }
22
23 std::cout << "[RSA] Server public key received: e=" <<
24     publicKey.e
25     << ", n=" << publicKey.n << "\n";
26
27 // 3. 客户端使用公钥加密 DES 会话密钥并发送。
28 if (!sendEncryptedDesKey(sockfd, desKey, publicKey))
29 {
30     throw std::runtime_error("send encrypted DES key
31     failed");
32 }
33
34 std::cout << "[RSA] DES session key encrypted and sent.\n";
35 return desKey;
36 }
```

Listing 10 客户端 RSA-DES 密钥交换关键代码

密钥交换完成后,服务端返回解密得到的 desKey,客户端返回自己生成的 desKey。二者内容一致,后续即可作为 DES 加密聊天的共享会话密钥。

3.5 DES 加密聊天功能复用

3.5.1 设计思路

本次实验复用第一次实验中已经实现并测试通过的聊天逻辑。第一次实验的聊天模块已经具备从标准输入读取明文、调用 encryptMessage() 加密、调用 sendPacket() 发送密文、调用 recvPacket() 接收密文、调用 decryptMessage() 解密显示以及处理 quit 退出指令等能力。

因此，本次实验只改变 DES 密钥的来源：第一次实验中密钥来自用户手工输入，本次实验中密钥来自 RSA-DES 自动密钥交换结果。密钥协商完成后，程序仍然复用原有 DES 加密聊天流程：

用户明文 → DES 加密 → TCP 密文传输 → DES 解密 → 明文显示 (40)

3.5.2 实验二主控流程对原聊天功能的复用

实验二主控模块位于 `rsa_des_chat.cpp`。服务端和客户端在完成 RSA-DES 密钥交换后,会根据用户选择的 I/O 模型进入 `secretChat()` 或 `secretChatSelect()`。

```
1 void runRsaDesServer(int port, ChatIoModel ioModel)
2 {
3     // 1. 创建监听 socket 并等待客户端连接。
4     int listenFd = createServerSocket(port);
5     std::string clientIp;
6     int clientPort = 0;
7     int connFd = acceptClient(listenFd, clientIp, clientPort);
8
9     closeSocket(listenFd);
10
11     // 2. 执行服务端 RSA-DES 自动密钥交换,得到 DES 会话密钥。
12     std::cout << "[RSA] Start RSA-DES key exchange as server.\n";
13     const std::string desKey =
14         serverPerformRsaDesKeyExchange(connFd);
15
16     // 3. 根据选择的 IO 模型进入 DES 加密聊天。
17     std::cout << "[CHAT] Begin DES encrypted chat.\n";
18
19     if (ioModel == ChatIoModel::SELECT)
20     {
21         secretChatSelect(connFd, clientIp, desKey,
22             AlgorithmType::DES);
23     }
24     else
25     {
26         secretChat(connFd, clientIp, desKey, AlgorithmType::DES);
27     }
```

```

26
27 // 4. 聊天结束后关闭通信 socket。
28 closeSocket(connFd);
29 }
30
31 void runRsaDesClient(const std::string& serverIp,
32                     int port,
33                     ChatIoModel ioModel)
34 {
35 // 1. 客户端连接服务端。
36 int sockfd = createClientSocket(serverIp, port);
37
38 // 2. 执行客户端 RSA-DES 自动密钥交换, 得到 DES 会话密钥。
39 std::cout << "[RSA] Start RSA-DES key exchange as client.\n";
40 const std::string desKey =
41     clientPerformRsaDesKeyExchange(sockfd);
42
43 // 3. 根据选择的 IO 模型进入 DES 加密聊天。
44 std::cout << "[CHAT] Begin DES encrypted chat.\n";
45
46 if (ioModel == ChatIoModel::SELECT)
47 {
48     secretChatSelect(sockfd, serverIp, desKey,
49                     AlgorithmType::DES);
50 }
51 else
52 {
53     secretChat(sockfd, serverIp, desKey, AlgorithmType::DES);
54 }
55
56 // 4. 聊天结束后关闭 socket。
57 closeSocket(sockfd);
58 }

```

Listing 11 实验二主控流程复用 DES 加密聊天功能

上述代码体现了本实验的主要复用思想：RSA-DES 密钥交换只发生在聊天开始之前，一旦双方获得相同的 DES 会话密钥，后续聊天流程即可直接复用第一次实验已有模块。

3.5.3 DES 加密和解密函数复用

第一次实验中, `encryptMessage()` 和 `decryptMessage()` 已经实现了统一的算法分发。本次实验主线使用 DES, 因此调用时传入 `AlgorithmType::DES`。报告中只展示 DES 分支, 省略 3DES 和 AES-128 分支。

```
1 std::vector<unsigned char> encryptMessage(const std::string&
    message,
2                                     const std::string& key,
3                                     AlgorithmType
    algorithmType)
4 {
5     switch (algorithmType)
6     {
7         case AlgorithmType::DES:
8         {
9             // 1. 创建 DES 对象并设置 RSA-DES 协商得到的会话密钥。
10            DES des;
11            if (!des.setKey(key))
12            {
13                throw std::runtime_error("DES setKey() failed:
invalid key");
14            }
15
16            // 2. 将明文字符串转换为字节数组。
17            std::vector<unsigned char> plainBytes =
stringToBytes(message);
18
19            // 3. 调用 DES 模块加密, 得到密文字节数组。
20            std::vector<unsigned char> cipherBytes =
des.encrypt(plainBytes);
21
22            if (cipherBytes.empty() && !plainBytes.empty())
23            {
24                throw std::runtime_error("DES encrypt() failed");
25            }
26
27            return cipherBytes;
28        }
29    }
```

```
29
30     default:
31         throw std::runtime_error("Unknown algorithm type in
encryptMessage()");
32     }
33 }
34
35 std::string decryptMessage(const std::vector<unsigned char>&
cipher,
36                             const std::string& key,
37                             AlgorithmType algorithmType)
38 {
39     switch (algorithmType)
40     {
41         case AlgorithmType::DES:
42         {
43             // 1. 创建 DES 对象并设置同一会话密钥。
44             DES des;
45             if (!des.setKey(key))
46             {
47                 throw std::runtime_error("DES setKey() failed:
invalid key");
48             }
49
50             // 2. 对收到的密文字节数组进行 DES 解密。
51             std::vector<unsigned char> plainBytes =
des.decrypt(cipher);
52
53             // 3. 解密失败时抛出异常。
54             if (cipher.size() > 0 && plainBytes.empty())
55             {
56                 throw std::runtime_error("DES decrypt() failed");
57             }
58
59             // 4. 将明文字节数组恢复为字符串。
60             return bytesToString(plainBytes);
61         }
62
63         default:
```

```

64         throw std::runtime_error("Unknown algorithm type in
        decryptMessage()");
65     }
66 }

```

Listing 12 DES 加密与解密分支关键代码

3.5.4 应用层封包机制复用

聊天密文仍然通过第一次实验实现的 `sendPacket()` 和 `recvPacket()` 发送与接收。该机制使用 4 字节长度头解决 TCP 字节流中的粘包、拆包和接收不完整问题。

```

1  bool sendPacket(int sockfd, const std::vector<unsigned char>&
    packet)
2  {
3      // 1. 将包体长度转换为 4 字节网络字节序长度头。
4      std::uint32_t bodyLength =
        static_cast<std::uint32_t>(packet.size());
5      std::uint32_t networkLength = hostToNetwork32(bodyLength);
6
7      unsigned char header[LENGTH_HEADER_SIZE];
8      std::memcpy(header, &networkLength, LENGTH_HEADER_SIZE);
9
10     // 2. 先完整发送长度头。
11     if (!sendAll(sockfd, header, LENGTH_HEADER_SIZE))
12     {
13         return false;
14     }
15
16     // 3. 再完整发送密文包体。
17     if (!packet.empty())
18     {
19         if (!sendAll(sockfd, packet.data(), packet.size()))
20         {
21             return false;
22         }
23     }
24
25     return true;

```

```
26 }
27
28 bool recvPacket(int sockfd, std::vector<unsigned char>& packet)
29 {
30     packet.clear();
31
32     // 1. 先接收 4 字节长度头。
33     unsigned char header[LENGTH_HEADER_SIZE] = {0};
34     if (!recvAll(sockfd, header, LENGTH_HEADER_SIZE))
35     {
36         return false;
37     }
38
39     // 2. 将网络字节序长度转换为主机字节序。
40     std::uint32_t networkLength = 0;
41     std::memcpy(&networkLength, header, LENGTH_HEADER_SIZE);
42     std::uint32_t bodyLength = networkToHost32(networkLength);
43
44     // 3. 按长度接收完整包体。
45     packet.resize(bodyLength);
46     if (bodyLength > 0 && !recvAll(sockfd, packet.data(),
47         packet.size()))
48     {
49         packet.clear();
50         return false;
51     }
52     return true;
53 }
```

Listing 13 应用层封包发送与接收关键代码

通过复用该封包机制，RSA 公钥包、RSA 加密后的 DES 密钥包和 DES 聊天密文包都能够稳定传输，从而保证程序整体通信可靠性。

3.6 基于 select 的全双工通信实现

3.6.1 设计思路

第一次实验中使用 `fork()` 实现全双工通信：父进程负责接收，子进程负责发送。该方式能够满足基本全双工要求，但需要创建子进程，并涉及子进程退出、父进程回收以及连接关闭等问题。

本次实验在保留 `fork()` 版本的基础上，新增了 `select()` 版本的单进程全双工通信函数 `secretChatSelect()`。该函数位于 `chat_select.cpp` 中，主要思想是将标准输入 `STDIN_FILENO` 和网络 `socket` 同时加入 `fd_set`，交给 `select()` 统一监听：

$$F = \{fd_{stdin}, fd_{socket}\} \quad (41)$$

当标准输入就绪时，程序读取用户输入、DES 加密并发送；当 `socket` 就绪时，程序接收密文包、DES 解密并显示。这样可以在单进程中实现双向通信。

3.6.2 select 全双工通信关键实现

```

1 void secretChatSelect(int sockfd,
2                       const std::string& peerName,
3                       const std::string& key,
4                       AlgorithmType algorithmType)
5 {
6     std::cout << "[CHAT] select model enabled.\n";
7     std::cout << "[CHAT] Type message and press Enter. Type quit
8         to exit.\n";
9
10    while (true)
11    {
12        // 1. 初始化 fd_set, 将标准输入和 socket 都加入监听集合。
13        fd_set readfds;
14        FD_ZERO(&readfds);
15        FD_SET(STDIN_FILENO, &readfds);
16        FD_SET(sockfd, &readfds);

```

```
16
17     const int maxFd = std::max(STDIN_FILENO, sockfd);
18
19     // 2. 使用 select 阻塞等待任意一个描述符变为可读。
20     const int ret = ::select(maxFd + 1, &readfds, nullptr,
21                               nullptr, nullptr);
22
23     if (ret < 0)
24     {
25         if (errno == EINTR)
26         {
27             continue;
28         }
29
30         throw std::runtime_error(
31             std::string("select() failed: ") +
32             std::strerror(errno)
33         );
34     }
35
36     // 3. 若 socket 可读, 说明对端发来了密文包。
37     if (FD_ISSET(sockfd, &readfds))
38     {
39         std::vector<unsigned char> cipherPacket;
40
41         if (!recvPacket(sockfd, cipherPacket))
42         {
43             std::cout << "[INFO] Peer disconnected or
44             receive failed. Chat ended.\n";
45             break;
46         }
47
48         std::string plainText =
49             decryptMessage(cipherPacket, key, algorithmType);
50
51         plainText = trimNewline(plainText);
52
53         if (!plainText.empty())
54         {
55
```



```
52         std::cout << "Receive message form <" <<
peerName << ">: "
53             << plainText << "\n";
54     }
55
56     if (isQuitCommand(plainText))
57     {
58         std::cout << "Quit!\n";
59         break;
60     }
61 }
62
63 // 4. 若标准输入可读，说明用户输入了待发送明文。
64 if (FD_ISSET(STDIN_FILENO, &readfds))
65 {
66     std::string inputLine;
67
68     if (!std::getline(std::cin, inputLine))
69     {
70         std::cout << "[INFO] Standard input closed. Stop
chatting.\n";
71         break;
72     }
73
74     inputLine = trimNewline(inputLine);
75
76     if (inputLine.empty())
77     {
78         continue;
79     }
80
81     std::vector<unsigned char> cipherPacket =
82         encryptMessage(inputLine, key, algorithmType);
83
84     if (!sendPacket(sockfd, cipherPacket))
85     {
86         std::cerr << "[ERROR] sendPacket() failed.\n";
87         break;
88     }
```

```
89
90         if (isQuitCommand(inputLine))
91         {
92             std::cout << "Quit!\n";
93             break;
94         }
95     }
96 }
97
98 // 5. 聊天结束后关闭双向通信，通知对端退出。
99 if (sockfd >= 0)
100 {
101     ::shutdown(sockfd, SHUT_RDWR);
102 }
103 }
```

Listing 14 select 全双工聊天关键代码

3.6.3 实现效果分析

该实现中，`select()` 每轮循环同时监听标准输入和 socket。由于两个输入源都由同一个进程统一管理，因此程序不需要额外创建子进程，也不需要处理父子进程同步问题。

从通信行为看：

1. 当用户键盘输入可读时，程序读取明文，调用 `encryptMessage()` 使用 DES 会话密钥加密，再调用 `sendPacket()` 发送密文。
2. 当 socket 可读时，程序调用 `recvPacket()` 接收完整密文包，再调用 `decryptMessage()` 解密并显示明文。
3. 当任意一方输入 quit 或连接关闭时，程序跳出循环，并通过 `shutdown(sockfd, SHUT_RDWR)` 关闭双向通信。

因此，`select()` 版本既满足实验要求中的全双工通信，又相比 `fork()` 版本减少了进程创建和回收开销，使通信逻辑更加集中清晰。该部分也体现了本实验在完成基

础 RSA-DES 自动密钥分配功能之外，对 Linux I/O 多路复用机制进行了进一步扩展实践。

四、程序运行结果与测试分析

本节对本实验程序的编译结果、RSA 算法正确性、RSA-DES 自动密钥交换流程、服务端与客户端实际运行效果、全双工聊天过程、quit 退出机制以及 fork() 与 select() 两种通信模型进行测试与分析。测试过程主要围绕以下目标展开：

1. 验证实验二新增代码能够在 Linux 环境下正常编译。
2. 验证 RSA 基础数学函数、RSA 加解密流程和 DES 密钥 RSA 加解密过程正确。
3. 验证客户端自动生成的 DES 会话密钥能够通过 RSA 公钥安全发送给服务端。
4. 验证服务端和客户端能够在无需手动输入 DES 密钥的情况下进入加密聊天。
5. 验证程序能够实现全双工通信，并能正常处理 quit 退出指令。
6. 对比 fork() 模型和 select() 模型在本实验中的运行效果和实现特点。

4.1 编译与运行环境

本实验程序在 Linux 操作系统下完成编译与运行。项目使用 Makefile 统一管理编译过程，实验二相关目标主要包括 rsa_chat、test_rsa 和 test_key_exchange。其中，rsa_chat 为本次实验的 RSA-DES 自动密钥分配聊天程序，test_rsa 用于验证 RSA 算法模块，test_key_exchange 用于验证 RSA-DES 自动密钥交换流程。

实验环境如表 7 所示。

表 7: 编译与运行环境

项目	实际环境	说明
操作系统	Ubuntu 24.04.3	程序基于 Linux Socket 和 <code>select()</code> 机制实现。
编译器	<code>g++ 13.3.0</code>	使用 C++17 标准编译。
编译标准	<code>-std=c++17</code>	支持 <code>std::thread</code> 、 <code>std::random_device</code> 等标准库功能。
构建工具	<code>make</code>	使用项目根目录下的 Makefile 统一构建。
测试端口	9004 / 9005	9004 用于 <code>select()</code> 模型测试，9005 用于 <code>fork()</code> 模型测试。
通信方式	TCP 一对一通信	服务端监听端口，客户端连接服务端。
加密方式	RSA-DES 混合加密	RSA 分发 DES 会话密钥，DES 加密聊天正文。

在项目根目录下执行如下命令进行编译：

```
1 make clean
2 make
```

Listing 15 完整编译命令

编译成功后，`bin` 目录下生成以下可执行文件：

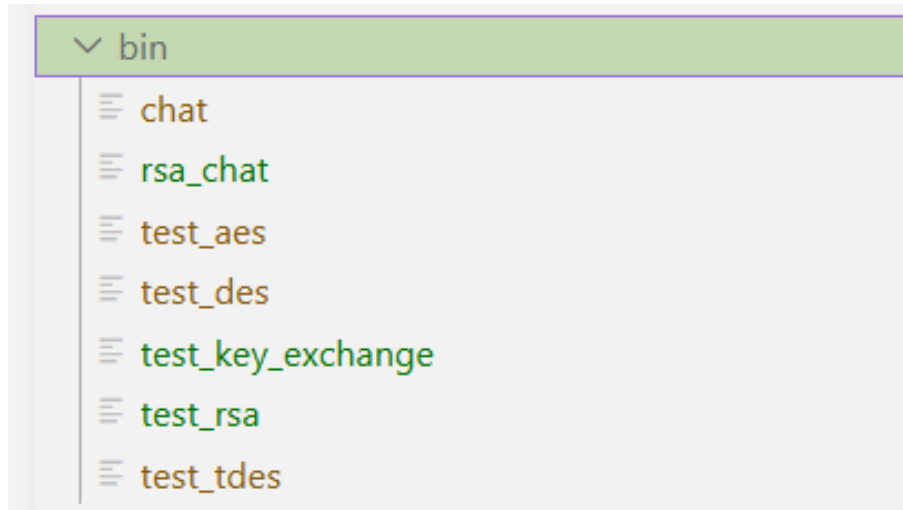


图 15: 实验二程序编译成功产物

从编译结果可以看出，实验二新增的 RSA 算法模块、RSA-DES 密钥交换模块、`select()` 聊天模块、实验二主控模块以及对应测试程序均能够通过编译，说明新增文件之间的接口衔接正确，原有 DES、TCP 和工具模块也能够被正常复用。

4.2 RSA 单元测试

RSA 单元测试程序为 `test/test_rsa.cpp`，编译生成的可执行文件为 `bin/test_rsa`。该测试程序主要验证 RSA 模块中的基础数学函数和 RSA 加解密功能是否正确，包括最大公约数、模逆元、快速模幂、素性测试、RSA 整数加解密以及 DES 会话密钥的 RSA 加解密。

运行命令如下：

```
1 ./bin/test_rsa
```

Listing 16 RSA 单元测试运行命令

运行结果如下：

```
● bryce_li@ltq:/mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/exp1_des_tcp_chat2.0$ ./bin/test_rsa

-----
[TEST GROUP] gcd / modInverse 测试
[PASS] gcd64(48,18)=6
[PASS] 17 与 3120 互素
[PASS] 17 关于 3120 的逆元为 2753
[PASS] e*d mod phi = 1
-----

[TEST GROUP] 快速模幂测试
[PASS] 4^13 mod 497 = 445
[PASS] 5^0 mod 19 = 1
[PASS] 2^10 mod 1000 = 24
-----

[TEST GROUP] Miller-Rabin 素性测试
[PASS] 65537 应识别为素数
[PASS] 32771 应识别为素数
[PASS] 65535 应识别为合数
[PASS] 100000 应识别为合数
-----

[TEST GROUP] RSA 整数加解密往返测试
[PASS] RSA 密钥对生成成功
[PASS] RSA integer round trip: 1
[PASS] RSA integer round trip: 65
[PASS] RSA integer round trip: 1024
[PASS] RSA integer round trip: 4096
[PASS] RSA integer round trip: 65535
-----

[TEST GROUP] DES 密钥 RSA 加解密测试
[PASS] RSA 密钥对生成成功
[PASS] 测试 DES 密钥长度为 8 字节
[PASS] 8 字节 DES 密钥拆成 4 个 RSA 分组
[PASS] DES 密钥经过 RSA 加解密后完全恢复
-----

[SUMMARY] passed = 21, failed = 0
```

图 16: RSA 单元测试通过结果

RSA 单元测试内容如表 8 所示。

表 8: RSA 单元测试项目

测试项目	测试目的	测试结果
<code>gcd64()</code> 测试	验证欧几里得算法能否正确计算最大公约数，并用于判断 e 与 $\varphi(n)$ 是否互素。	通过
<code>modInverse()</code> 测试	验证扩展欧几里得算法能否正确求出模逆元，例如 17 关于 3120 的逆元为 2753。	通过
<code>modPow()</code> 测试	验证快速模幂函数能否正确计算 $a^b \bmod n$ ，确保 RSA 加解密核心运算正确。	通过
素性测试	验证 Miller-Rabin 素性检测函数能够识别素数和合数。	通过
RSA 整数往返测试	验证整数明文经过 RSA 公钥加密、私钥解密后能够恢复原始数据。	通过
DES 密钥 RSA 加解密测试	验证 8 字节 DES 密钥拆分为 4 个 RSA 分组后，加密和解密结果能够完全恢复原密钥。	通过

测试结果中所有测试项均显示 [PASS]，且最终汇总信息中 `failed = 0`，说明 RSA 模块的关键数学函数和加解密流程均能正确工作。其中，DES 密钥 RSA 加解密测试表明，本实验将 8 字节 DES 会话密钥拆成 4 个 16 bit 分组后分别进行 RSA 加密和解密，能够完整恢复出原始 DES 密钥。这为后续 RSA-DES 自动密钥交换提供了算法正确性基础。

4.3 自动密钥交换测试

自动密钥交换测试程序为 `test/test_key_exchange.cpp`，编译生成的可执行文件为 `bin/test_key_exchange`。该测试使用 `socketpair()` 在本机模拟一条全双工连接，并分别启动服务端线程和客户端线程执行真实的 RSA-DES 密钥交换流程。

运行命令如下：

```
1 ./bin/test_key_exchange
```

Listing 17 自动密钥交换测试运行命令

该测试主要验证如下过程：

1. 服务端线程生成 RSA 公钥对。
2. 服务端线程发送 RSA 公钥。
3. 客户端线程自动生成 8 字节 DES 会话密钥。
4. 客户端线程接收 RSA 公钥，并使用该公钥加密 DES 会话密钥。
5. 客户端线程发送 RSA 加密后的 DES 密钥。
6. 服务端线程使用 RSA 私钥解密，恢复 DES 会话密钥。
7. 测试程序比较服务端恢复出的 DES 密钥与客户端生成的 DES 密钥是否完全一致。

测试结果如下：

```
● bryce_li@ltq: /mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/exp1_des_tcp_chat2.0$ ./bin/test_key_exchange
-----
[TEST GROUP] RSA-DES 自动密钥交换测试
[RSA] Key pair generated.
[DES] Random DES session key generated. [hidden]
[RSA] Public key: e=65537, n=1899348289
[RSA] Private key is kept only on server. [hidden]
[RSA] Public key sent to client.
[RSA] Server public key received: e=65537, n=1899348289
[RSA] DES session key encrypted and sent.
[RSA] DES session key decrypted successfully. [hidden]
[PASS] 服务端密钥交换流程无异常
[PASS] 客户端密钥交换流程无异常
[PASS] 服务端解密得到合法 DES 密钥
[PASS] 客户端生成合法 DES 密钥
[PASS] 服务端与客户端最终 DES 会话密钥一致
-----
[SUMMARY] passed = 5, failed = 0
```

图 17: RSA-DES 自动密钥交换测试结果

自动密钥交换测试结果记录如表 9 所示。

表 9: RSA-DES 自动密钥交换测试项目

测试项目	验证内容	测试结果
服务端密钥交换流程	验证服务端能够生成 RSA 密钥对、发送公钥、接收密钥密文并完成私钥解密。	通过
客户端密钥交换流程	验证客户端能够自动生成 DES 密钥、接收 RSA 公钥、加密并发送 DES 密钥。	通过
服务端 DES 密钥合法性	验证服务端解密得到的 DES 会话密钥长度为 8 字节，符合 DES 模块要求。	通过
客户端 DES 密钥合法性	验证客户端自动生成的 DES 会话密钥符合 DES 密钥长度要求。	通过
双方 DES 密钥一致性	验证服务端最终得到的 DES 密钥与客户端生成的 DES 密钥完全一致。	通过

从测试结果可以看出，客户端自动生成的 DES 会话密钥经过 RSA 公钥加密发送后，服务端能够使用 RSA 私钥正确解密恢复，并且双方最终持有的 DES 会话密钥完全一致。该测试证明本实验的密钥自动生成、RSA 公钥分发、RSA 加密传输和 RSA 私钥解密流程整体正确。

4.4 服务端运行截图

完成单元测试和密钥交换测试后，进一步进行真实 TCP 通信场景下的服务端与客户端运行测试。首先打开终端 A，启动服务端。以 `select()` 通信模型为例，服务端运行命令如下：

```
1 ./bin/rsa_chat -m server -p 9004 --io select
```

Listing 18 服务端启动命令

服务端启动后，应首先创建监听 socket，并等待客户端连接。当客户端连接成功后，

服务端会执行 RSA-DES 密钥交换流程，包括生成 RSA 公私钥、发送 RSA 公钥、接收客户端发来的 DES 会话密钥密文、使用 RSA 私钥解密，并最终进入 DES 加密聊天阶段。

```
bryce_li@ltq:/mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/
exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m server -p 9004 --io select
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Server
Port      : 9004
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : select
=====

[INFO] Starting Lab2 RSA-DES server...
[TCP] Listening on port 9004 ...
```

图 18: 服务端启动并监听端口

```
[TCP] Client connected from 127.0.0.1:47188
[RSA] Start RSA-DES key exchange as server.
[RSA] Key pair generated.
[RSA] Public key: e=65537, n=3607200689
[RSA] Private key is kept only on server. [hidden]
[RSA] Public key sent to client.
[RSA] DES session key decrypted successfully. [hidden]
[CHAT] Begin DES encrypted chat.
[CHAT] select model enabled.
[CHAT] Type message and press Enter. Type quit to exit.
□
```

图 19: 服务端完成 RSA-DES 密钥交换

服务端关键运行状态如表 10 所示。

表 10: 服务端运行状态记录

阶段	预期现象	实际结果
启动监听	服务端成功监听指定端口 9004。	[TCP] Listening on port 9004 ...
客户端连接	服务端显示客户端 IP 和端口信息。	[TCP] Client connected from 127.0.0.1:47188
RSA 密钥生成	服务端生成 RSA 公钥和私钥，并显示公钥参数。	[RSA] Public key: e=65537, n=3607200689
公钥发送	服务端向客户端发送 RSA 公钥。	[RSA] Public key sent to client.
DES 密钥解密	服务端接收 RSA 加密后的 DES 密钥，并使用私钥解密成功。	[RSA] DES session key decrypted successfully. [hidden]
进入聊天	服务端进入 DES 加密聊天状态，并启用 select() 模型。	[CHAT] Begin DES encrypted chat. [CHAT] select model enabled.

服务端运行结果表明，服务端能够正确完成 TCP 监听、客户端接入、RSA 密钥生成、公钥发送和 DES 会话密钥解密过程。需要注意的是，服务端只显示 RSA 公钥参数，私钥不会通过网络发送，也不会正式运行中直接暴露，从而符合 RSA 公钥密码体制的安全设计原则。

4.5 客户端运行截图

在服务端启动并监听端口后，打开终端 B 启动客户端。以本机回环地址测试为例，客户端运行命令如下：

```
1 ./bin/rsa_chat -m client -i 127.0.0.1 -p 9004 --io select
```

Listing 19 客户端启动命令

客户端连接服务端后，会自动生成 8 字节 DES 会话密钥，随后接收服务端 RSA 公钥，并使用该公钥加密 DES 会话密钥发送给服务端。密钥交换完成后，客户端进入 DES 加密聊天阶段。

客户端关键运行状态如表 11 所示。

表 11: 客户端运行状态记录

阶段	预期现象	实际结果
连接服务端	客户端成功连接 127.0.0.1:9004。	[TCP] Connect Success.
自动生成 DES 密钥	客户端提示随机 DES 会话密钥已生成，但不显示密钥明文。	[DES] Random DES session key generated. [hidden]
接收 RSA 公钥	客户端显示接收到服务端 RSA 公钥参数 e, n 。	[RSA] Server public key received: e=65537, n=3607200689
加密发送 DES 密钥	客户端使用 RSA 公钥加密 DES 密钥并发送。	[RSA] DES session key encrypted and sent.
进入聊天	客户端进入 DES 加密聊天状态，启用 <code>select()</code> 模型。	[CHAT] Begin DES encrypted chat. [CHAT] select model enabled.

```
○ bryce_li@ltq: /mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m client -i 127.0.0.1 -p 9004 --io select
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Client
Port      : 9004
Server IP : 127.0.0.1
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : select
=====

[INFO] Starting Lab2 RSA-DES client...
[TCP] Connect Success.
[RSA] Start RSA-DES key exchange as client.
[DES] Random DES session key generated. [hidden]
[RSA] Server public key received: e=65537, n=3607200689
[RSA] DES session key encrypted and sent.
[CHAT] Begin DES encrypted chat.
[CHAT] select model enabled.
[CHAT] Type message and press Enter. Type quit to exit.
□
```

图 20: 客户端完成 RSA-DES 密钥交换

客户端运行结果表明，用户不再需要通过 `-k` 参数或交互输入方式提供 DES 密钥。DES 会话密钥由客户端自动生成，并通过 RSA 公钥加密后发送给服务端。整个过程对用户透明，用户只需要关注聊天内容本身。

4.6 全双工聊天测试

完成 RSA-DES 密钥交换后，服务端和客户端进入 DES 加密聊天阶段。在该阶段，双方均使用协商得到的同一 DES 会话密钥对聊天消息进行加密和解密。用户在终端输入的是明文，程序发送前自动调用 DES 加密函数，接收端收到密文后自动调用 DES 解密函数并显示明文。

本实验使用 `select()` 模型进行全双工聊天测试。测试过程中，客户端和服务端分别发送多条消息，验证双方均能正常发送和接收。

聊天测试如下：

```

bryce_li@ltq:/mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m client -i 127.0.0.1 -p 9004 --io select
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Client
Port      : 9004
Server IP : 127.0.0.1
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : select
=====

[INFO] Starting Lab2 RSA-DES client...
[TCP] Connect Success.
[RSA] Start RSA-DES key exchange as client.
[DES] Random DES session key generated. [hidden]
[RSA] Server public key received: e=65537, n=3607200689
[RSA] DES session key encrypted and sent.
[CHAT] Begin DES encrypted chat.
[CHAT] select model enabled.
[CHAT] Type message and press Enter. Type quit to exit.
hello server
Receive message form <127.0.0.1>: hello client
Receive message form <127.0.0.1>: 你好客户端
你好服务器
123456@#¥%
Receive message form <127.0.0.1>: 123456@#¥%
□

```

图 21: 客户端聊天测试

```

bryce_li@ltq:/mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/
exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m server -p 9004 --io select
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Server
Port      : 9004
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : select
=====

[INFO] Starting Lab2 RSA-DES server...
[TCP] Listening on port 9004 ...
[TCP] Client connected from 127.0.0.1:47188
[RSA] Start RSA-DES key exchange as server.
[RSA] Key pair generated.
[RSA] Public key: e=65537, n=3607200689
[RSA] Private key is kept only on server. [hidden]
[RSA] Public key sent to client.
[RSA] DES session key decrypted successfully. [hidden]
[CHAT] Begin DES encrypted chat.
[CHAT] select model enabled.
[CHAT] Type message and press Enter. Type quit to exit.
Receive message form <127.0.0.1>: hello server
hello client
你好客户端
Receive message form <127.0.0.1>: 你好服务器
Receive message form <127.0.0.1>: 123456@#¥%
123456@#¥%
□

```

图 22: 服务端聊天测试

测试用例如表 12 所示。

表 12: 全双工聊天测试记录

序号	发送方	发送明文	接收方显示内容	测试结果
1	客户端	hello server	Receive message form <127.0.0.1>: hello server	通过
2	服务端	hello client	Receive message form <127.0.0.1>: hello client	通过
3	客户端	你好服务器	Receive message form <127.0.0.1>: 你好服务器	通过
4	服务端	你好客户端	Receive message form <127.0.0.1>: 你好客户端	通过
5	双方交替发送	123456@ \$ %	Receive message form <127.0.0.1>: 123456@ \$ %	通过

测试结果表明，RSA-DES 密钥交换完成后，双方能够正常进入 DES 加密聊天阶段。客户端和服务端均可以发送消息，也可以接收对方消息，说明程序满足全双工通信要求。用户输入和终端显示均为明文，但程序内部已经完成发送前 DES 加密和接收后 DES 解密，因此加密过程对用户透明。

4.7 quit 退出测试

为了验证程序的正常退出机制，本实验进一步测试 quit 指令。当任意一方输入 quit 后，该消息会按照普通聊天消息流程先进行 DES 加密，再通过 TCP 发送给对端。接收端解密后识别到 quit 指令，随后结束聊天循环并关闭连接。

测试命令和操作如下：

```
1 # 在服务端或客户端任意一端输入：
2 quit
```

Listing 20 quit 退出测试操作

退出测试结果如表 13 所示。

表 13: quit 退出测试记录

测试场景	操作	预期现象	测试结果
客户端退出	客户端输入 quit	客户端显示退出提示，服务端接收到 quit 后结束聊天。	两端均显示 Quit! 并成功结束聊天
服务端退出	服务端输入 quit	服务端显示退出提示，客户端接收到 quit 后结束聊天。	两端均显示 Quit! 并成功结束聊天

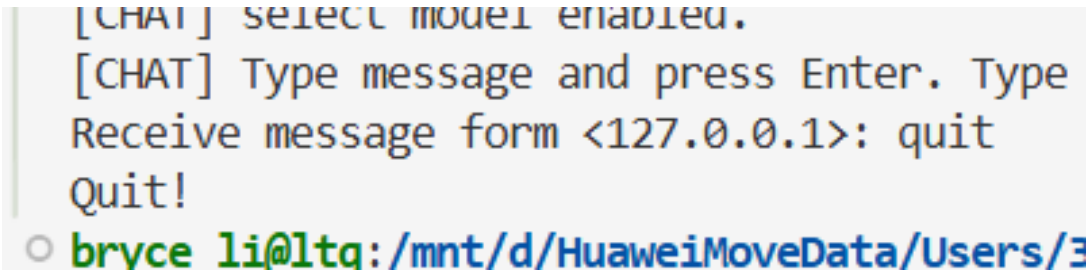


图 23: quit 指令正常退出测试

测试结果表明，quit 指令能够被正常加密发送、接收解密和识别处理。程序在退出时能够跳出聊天循环并关闭 socket 连接，说明退出机制与 DES 加密聊天流程能够正确结合。

4.8 fork 与 select 模型对比

第一次实验中，程序使用 `fork()` 实现全双工通信，即父进程和子进程分别负责接收和发送。本次实验在保留 `fork()` 模型的基础上，新增 `select()` 模型。为了验证两种模型均能正常运行，本实验分别使用不同端口进行测试。

`select()` 模型测试命令如下：

```
1 # 终端 A: 服务端
2 ./bin/rsa_chat -m server -p 9004 --io select
3
4 # 终端 B: 客户端
5 ./bin/rsa_chat -m client -i 127.0.0.1 -p 9004 --io select
```

Listing 21 select 模型测试命令

`fork()` 模型测试命令如下：

```
1 # 终端 A: 服务端
2 ./bin/rsa_chat -m server -p 9005 --io fork
3
4 # 终端 B: 客户端
5 ./bin/rsa_chat -m client -i 127.0.0.1 -p 9005 --io fork
```

Listing 22 fork 模型测试命令

两种通信模型对比如表 14 所示。

表 14: fork() 模型与 select() 模型对比

对比项	fork() 模型	select() 模型
实现方式	创建子进程, 通常父进程负责接收, 子进程负责发送。	单进程同时监听标准输入和 socket, 根据就绪事件执行发送或接收。
进程数量	每个聊天端需要额外创建子进程。	单进程即可完成全双工通信。
资源开销	需要进程创建和进程回收, 资源开销相对较大。	不需要创建额外进程, 资源开销相对较小。
退出处理	需要处理父子进程退出、信号通知和进程回收。	只需跳出事件循环并关闭 socket, 逻辑更集中。
程序结构	发送逻辑和接收逻辑分布在不同进程中。	发送和接收逻辑统一在一个事件循环中处理。
全双工能力	能够实现全双工通信。	能够实现全双工通信。
本实验定位	保留第一次实验已有成果, 用于对比验证。	本次实验扩展提高内容, 体现 Linux I/O 多路复用机制。

```

bryce_li@ltq: /mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m server -p 9005 --io fork
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Server
Port      : 9005
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : fork
=====

[INFO] Starting Lab2 RSA-DES server...
[TCP] Listening on port 9005 ...
[TCP] Client connected from 127.0.0.1:56240
[RSA] Start RSA-DES key exchange as server.
[RSA] Key pair generated.
[RSA] Public key: e=65537, n=2373572843
[RSA] Private key is kept only on server. [hidden]
[RSA] Public key sent to client.
[RSA] DES session key decrypted successfully. [hidden]
[CHAT] Begin DES encrypted chat.

```

图 24: fork() 模式下 RSA-DES 模式正确运行

```
bryce_li@ltq:/mnt/d/HuaweiMoveData/Users/33537/Desktop/网络安全技术/
● exp1_des_tcp_chat2.0$ ./bin/rsa_chat -m server -p 9004 --io select
=====
Secure TCP Chat Program - Lab2 RSA-DES
RSA distributes DES session key automatically
=====

===== Lab2 Configuration Summary =====
Mode      : Server
Port      : 9004
Algorithm : DES after RSA key exchange
Key       : generated automatically [hidden]
IO Model  : select
=====

[INFO] Starting Lab2 RSA-DES server...
[TCP] Listening on port 9004 ...
[TCP] Client connected from 127.0.0.1:47188
[RSA] Start RSA-DES key exchange as server.
[RSA] Key pair generated.
[RSA] Public key: e=65537, n=3607200689
[RSA] Private key is kept only on server. [hidden]
[RSA] Public key sent to client.
[RSA] DES session key decrypted successfully. [hidden]
[CHAT] Begin DES encrypted chat.
[CHAT] select model enabled.
[CHAT] Type message and press Enter. Type quit to exit.
```

图 25: select() 模型下 RSA-DES 模式正常运行

从测试结果可以看出，fork() 和 select() 两种模型均能够完成 RSA-DES 自动密钥交换后的全双工加密聊天。其中，fork() 模型延续了第一次实验的设计思路，逻辑直观，适合初步实现发送和接收分离；select() 模型则通过单进程同时监听标准输入和 socket，避免了额外进程创建和回收，程序结构更加集中。

五、扩展与提高

在完成 RSA 自动分发 DES 会话密钥和 DES 加密聊天功能的基础上，本节进一步从安全性、性能与实际适用性角度对实验方案进行分析。该部分主要用于说明本实验为什么采用 RSA-DES 混合加密结构，以及当前实验方案与真实安全通信系统之间仍然存在的差距。

5.1 RSA 分发 DES 会话密钥的安全性分析

1. 相比手工共享密钥的改进

第一次实验中，DES 密钥需要由服务端和客户端提前手工设置，并且双方必须输入完全相同的 8 字节密钥。这种方式虽然能够验证 DES 加密聊天流程，但在真实网络通信中存在明显问题：如果通信双方无法通过安全信道提前共享密钥，那么 DES 密钥本身就可能在传递过程中被窃听。

本次实验引入 RSA 后，密钥分发方式发生了变化。服务端生成 RSA 公私钥对，并将公钥发送给客户端；客户端自动生成 DES 会话密钥后，使用服务端公钥加密该密钥；服务端再使用本地私钥解密恢复 DES 会话密钥。整个过程中，DES 会话密钥不再以明文形式直接在网络上传输。

其安全性改进可以概括为：

1. DES 会话密钥由客户端自动生成，减少人工输入错误和人为泄露风险。
2. 网络中传输的是 RSA 加密后的 DES 密钥，而不是 DES 密钥明文。
3. RSA 私钥始终保存在服务端本地，不通过网络传输。
4. 即使攻击者截获加密后的 DES 密钥，在不知道 RSA 私钥的情况下也难以直接恢复会话密钥。

因此，本实验实现了从预共享对称密钥到公钥加密分发会话密钥的升级，更接近真实安全通信系统中的密钥协商思想。

2. RSA 公钥和私钥的作用划分

RSA 算法的安全使用依赖于公钥和私钥的明确分工。本实验中，公钥用于加密，私钥用于解密：

$$C \equiv M^e \pmod{n} \quad (42)$$

$$M \equiv C^d \pmod{n} \quad (43)$$

其中, (e, n) 为公钥, 可以发送给客户端; (d, n) 为私钥, 只能由服务端保存。程序中服务端只调用 `sendRsaPublicKey()` 发送公钥, 不会发送私钥。客户端也只需要 RSA 公钥即可完成 DES 会话密钥加密。

从安全角度看, 私钥不出现在网络中是 RSA 密钥分发方案成立的基础。如果将私钥发送给客户端或暴露在终端日志中, 攻击者一旦获得私钥, 就可以解密客户端发送的 DES 会话密钥, 后续聊天内容也将失去保密性。因此, 本实验在运行日志中仅显示 RSA 公钥参数, 私钥和 DES 会话密钥均以 [hidden] 形式隐藏。

3. 会话密钥机制的意义

本实验中客户端每次连接服务端时都会重新生成一个 DES 会话密钥。该密钥只用于当前 TCP 聊天连接, 连接结束后即失效。相比长期固定使用同一个 DES 密钥, 会话密钥机制具有更好的安全性。

其主要意义包括:

1. 不同会话使用不同 DES 密钥, 降低单次密钥泄露造成的影响范围。
2. 即使某一次通信密钥被破解, 也不会直接影响其他连接。
3. 会话密钥由程序自动生成, 避免用户长期复用弱口令式密钥。

因此, RSA 分发 DES 会话密钥不仅解决了密钥传输问题, 也使系统具备了基本的每次会话独立密钥特征。

5.2 混合加密方案的性能与适用性分析

1. 为什么不直接使用 RSA 加密聊天正文

RSA 属于非对称加密算法, 优点是能够解决密钥分发问题, 但其加解密运算涉及大整数模幂, 计算开销明显高于 DES 这类对称加密算法。如果直接使用 RSA 加密每一条聊天消息, 会带来以下问题:

- 1. RSA 加解密速度较慢，不适合频繁处理大量聊天正文。
- 2. RSA 明文分组大小受模数 n 限制，长消息需要拆分成大量小块。
- 3. 每条消息都使用 RSA 加解密会增加通信延迟和程序复杂度。

因此，本实验没有使用 RSA 直接加密聊天正文，而是只使用 RSA 加密一次 DES 会话密钥。密钥交换完成后，后续所有聊天内容都交给 DES 处理。

2. DES 与 RSA 的分工优势

本实验采用的 RSA-DES 混合加密方案充分利用了两类密码算法的特点：

表 15: RSA 与 DES 在本实验中的分工

算法	优势	本实验中的作用
RSA	公钥可以公开，能够解决对称密钥在网络中的安全分发问题。	服务端发送公钥，客户端用公钥加密 DES 会话密钥。
DES	加解密速度快，适合处理连续聊天数据。	双方使用协商得到的 DES 会话密钥加密和解密聊天正文。

这种设计体现了典型混合加密思想：

RSA 保护密钥，DES 保护数据

(44)

也就是说，RSA 只在连接建立初期参与密钥交换，DES 则在整个聊天阶段持续工作。这样既避免了手工共享密钥的问题，又避免了 RSA 直接加密大量数据带来的性能开销。

3. 本实验方案的适用场景

从实验角度看，本方案适用于一对一 TCP 聊天程序中的会话密钥自动分配场景。它能够较好地展示以下知识点：

- 1. 公钥密码体制如何解决对称密钥分发问题。

2. 对称加密和非对称加密如何组合使用。
3. TCP 连接建立后如何设计应用层密钥交换协议。
4. 密钥交换完成后如何复用原有 DES 加密通信模块。

从实际系统角度看，RSA-DES 混合结构也与许多安全通信协议的基本思想一致。例如，真实协议中通常不会使用非对称加密算法直接加密全部业务数据，而是用其协商或保护会话密钥，再使用对称算法加密大量数据。虽然本实验使用的是教学规模的 RSA 和 DES，但其整体设计思路具有较强的代表性。

5.3 实验方案的安全局限与改进方向

1. RSA 参数规模较小

为了便于实验实现和调试，本实验中的 RSA 使用 64 位整数进行运算，素数 p, q 的位数也较小。这种设计能够清晰展示 RSA 的数学流程，包括素数生成、计算 $\varphi(n)$ 、选择 e 、计算 d 、快速模幂加解密等步骤，但其安全强度不能与真实工程中的 RSA 相比。

真实安全系统中的 RSA 通常需要使用至少 2048 bit 甚至更长的密钥，并依赖成熟的大整数库完成运算。因此，本实验 RSA 模块更适合教学演示和原理验证，不应直接用于真实安全通信。

改进方向如下：

1. 引入大整数库，例如 GMP、OpenSSL BN 或其他可靠密码库。
2. 将 RSA 密钥长度提升到 2048 bit 或更高。
3. 使用标准填充方案，例如 OAEP，而不是直接对整数分组做裸 RSA 加密。

2. DES 算法本身安全性不足

DES 的密钥长度只有 56 bit，现代计算能力下已经不能认为是安全的。本实验使用 DES 是因为课程前一章已经实现了 DES 加密聊天程序，本次实验要求在其基础上扩展 RSA 自动密钥分配功能。因此，本实验中 DES 的主要作用是作为对称加密算法代表，用于展示会话密钥加密聊天正文的机制。

如果将本方案改进为更接近真实应用的安全通信系统，应使用更现代的对称加密算法，例如 AES。第一次实验项目中已经扩展实现了 AES-128，因此后续可以将 RSA 分发的会话密钥从 8 字节 DES 密钥扩展为 16 字节 AES-128 密钥，从而提高数据加密阶段的安全性。

可行改进包括：

1. 使用 AES-128 或 AES-256 替换 DES。
2. 避免 ECB 模式，改用 CBC、CTR 或 GCM 等更安全的工作模式。
3. 为每次会话或每条消息引入随机 IV，避免相同明文产生相同密文。

3. 缺少身份认证，可能受到中间人攻击

本实验中服务端会将 RSA 公钥发送给客户端，客户端收到公钥后直接使用它加密 DES 会话密钥。该过程虽然避免了 DES 密钥明文传输，但仍然存在一个重要问题：客户端没有验证收到的 RSA 公钥是否确实来自目标服务端。

如果攻击者位于客户端和服务端之间，可能拦截服务端公钥并替换成自己的公钥。客户端随后会使用攻击者的公钥加密 DES 会话密钥，攻击者即可用自己的私钥解密得到 DES 密钥，再重新用服务端公钥加密后转发给服务端。这样客户端和服务端都可能认为通信正常，但攻击者已经能够解密中间传输内容。

该问题属于典型的中间人攻击。要解决该问题，需要引入身份认证机制。

改进方向如下：

1. 为服务端 RSA 公钥绑定数字证书，由可信 CA 证明公钥归属。
2. 客户端内置或提前保存服务端公钥指纹，连接时进行指纹校验。
3. 使用数字签名机制验证服务端身份。
4. 参考 TLS 协议，引入完整的握手认证和密钥协商流程。

4. 缺少消息完整性和认证保护

当前实验主要关注机密性，即防止聊天明文和 DES 会话密钥直接暴露。但在真实安全通信中，仅加密还不够，还需要保证消息没有被篡改。当前方案中，如果攻击者篡改密文，接收端可能解密失败，也可能得到错误明文，但程序并没有对消息完整性进行严格认证。

后续可以在每个聊天数据包中加入消息认证码，例如 HMAC；或者直接使用带认证的加密模式，例如 AES-GCM。这样接收端不仅能解密消息，还能判断消息是否被篡改。

可行改进包括：

1. 为每个密文包添加 HMAC，接收端先验证再解密。
2. 使用 AES-GCM 等认证加密模式，同时提供保密性和完整性。
3. 在数据包中增加序号字段，防止重放攻击。

5. 总体改进思路

综合来看，本实验已经实现了 RSA 自动分发 DES 会话密钥的核心流程，能够满足课程实验对 RSA 原理、密钥共享、DES 加密聊天和全双工通信的要求。但若从真实安全通信系统角度进一步提升，可以按照如下方向改进：

表 16: 实验方案局限与改进方向

当前局限	影响	改进方向
RSA 密钥规模较小	只能用于实验演示，无法抵抗真实攻击。	使用大整数库和 2048 bit 以上 RSA 密钥。
使用裸 RSA 分组加密	可能存在确定性加密和填充安全问题。	使用 RSA-OAEP 等标准填充方案。
DES 安全性不足	56 bit 密钥强度较低。	使用 AES-128 或 AES-256 替代 DES。
缺少公钥身份认证	可能受到中间人攻击。	引入证书、数字签名或公钥指纹校验。
缺少消息完整性校验	无法可靠检测密文被篡改。	使用 HMAC 或 AES-GCM 等认证加密机制。

因此，本实验的价值主要在于清晰展示 RSA 与 DES 组合使用的基本思想，并通过实际代码完成自动密钥分发和加密聊天流程。通过分析其局限，也可以进一步理解真实安全协议为什么需要更大密钥、更安全的对称算法、身份认证、消息认证和标准化协议设计。

六、实验遇到的问题及其解决方法

在本次实验中，程序需要在第一次实验 DES 加密聊天程序的基础上进一步加入 RSA 自动密钥分发机制，并扩展实现 `select()` 全双工通信模型。相比第一次实验，本次实验的难点不只在于单个算法的实现，还在于 RSA、DES、TCP 封包协议和聊天主流程之间的正确衔接。下面选取实验过程中最关键的几个问题进行说明。

6.1 问题一：RSA 不能直接加密完整 8 字节 DES 密钥

1. 问题描述

本实验中 DES 会话密钥长度固定为 8 字节。如果直接将 8 字节密钥整体转换成一个整数并使用 RSA 加密，可能会出现明文整数 M 大于 RSA 模数 n 的情况。而 RSA 算法要求：

$$0 \leq M < n \quad (45)$$

若明文分组不满足该条件，则 RSA 加解密后的结果不能保证正确恢复原始数据。尤其在本实验中，为了避免引入额外大整数库，RSA 使用 64 位整数实现，实验规模下的 n 不适合直接承载完整 8 字节 DES 密钥。

2. 解决方法

为解决该问题，程序没有将 8 字节 DES 密钥整体作为一个大整数加密，而是将其拆分为 4 个 16 bit 分组：

$$K_{DES} = k_0 \parallel k_1 \parallel k_2 \parallel k_3 \quad (46)$$

其中每个 k_i 由 2 个字节组成，数值范围为 $0 \sim 65535$ 。在服务端 RSA 密钥生成时，选取的 p, q 能够保证 n 通常大于 16 bit 明文分组，因此每个分组均满足 RSA 加密条件。客户端分别对 4 个分组执行 RSA 公钥加密，服务端再分别使用 RSA 私钥解密并拼接回 8 字节 DES 会话密钥。

关键处理思路如下：

```

1 for (std::size_t i = 0; i < desKey.size(); i += 2)
2 {
3     // 1. 每两个字节合成为一个 16 bit 明文分组。
4     const std::uint64_t high =
5         static_cast<unsigned char>(desKey[i]);
6
7     const std::uint64_t low =

```

```

8         static_cast<unsigned char>(desKey[i + 1]));
9
10        const std::uint64_t plainChunk = (high << 8ULL) | low;
11
12        // 2. 对每个 16 bit 分组分别执行 RSA 加密。
13        cipherChunks.push_back(RSA::encryptInteger(plainChunk,
14            publicKey));
14    }

```

Listing 23 DES 密钥分组后再进行 RSA 加密

这样既避免了明文整数超过 RSA 模数的问题，也保持了 DES 密钥恢复过程的简单性和确定性。经过 `test_rsa` 和 `test_key_exchange` 测试验证，服务端解密得到的 DES 会话密钥能够与客户端生成的密钥完全一致。

6.2 问题二：RSA 公钥和加密密钥包不能直接发送结构体

1. 问题描述

在 RSA-DES 密钥交换过程中，服务端需要发送 RSA 公钥 e, n ，客户端需要发送 RSA 加密后的 DES 密钥分组。最初设计时容易想到直接定义结构体并通过 `send()` 发送结构体内存，例如：

```
1 send(sockfd, &publicKey, sizeof(publicKey), 0);
```

但这种方式存在隐患：

1. C++ 结构体可能存在内存填充字节，不同编译环境下结构体大小不一定完全一致。
2. 多字节整数在不同主机上的字节序可能不同。
3. TCP 是字节流协议，单次 `send()` 和 `recv()` 不能保证一次发送的数据被一次完整接收。
4. 缺少包类型标识，不利于区分 RSA 公钥包和 DES 密钥密文包。

如果直接发送结构体，可能导致接收端解析错误，甚至出现密钥交换失败。

2. 解决方法

本实验沿用第一次实验已经实现的应用层封包机制,即所有网络数据都通过 `sendPacket()` 和 `recvPacket()` 发送接收。该机制使用 4 字节长度头标识包体长度,从而解决 TCP 粘包、拆包和接收不完整问题。

在 RSA-DES 密钥交换模块中,程序进一步手动组织字节数组,而不是直接发送结构体内存。RSA 公钥包使用如下格式:

$$\text{RPK1} \parallel e \parallel n \quad (47)$$

加密 DES 密钥包使用如下格式:

$$\text{EDK1} \parallel \text{count} \parallel C_0 \parallel C_1 \parallel C_2 \parallel C_3 \quad (48)$$

其中,RPK1 和 EDK1 为 magic 字段,用于标识数据包类型。整数写入时统一采用固定长度和固定字节序,从而保证接收端能够稳定解析。

关键发送代码如下:

```

1 bool sendRsaPublicKey(int sockfd, const RsaPublicKey& publicKey)
2 {
3     // 1. 检查公钥参数。
4     if (publicKey.e == 0 || publicKey.n == 0)
5     {
6         return false;
7     }
8
9     // 2. 手动组织字节数组,避免直接发送结构体。
10    std::vector<unsigned char> packet;
11    packet.reserve(20);
12
13    appendMagic(packet, RSA_PUBLIC_KEY_MAGIC);
14    appendUint64(packet, publicKey.e);
15    appendUint64(packet, publicKey.n);
16
17    // 3. 复用第一次实验的 sendPacket(), 保证完整发送。

```

```

18     return sendPacket(sockfd, packet);
19 }

```

Listing 24 手动组织 RSA 公钥包并复用封包发送

通过这种方式，RSA-DES 密钥交换数据包和后续 DES 聊天密文包都使用统一的长度头封包协议，增强了程序的可靠性和可维护性。

6.3 问题三：从 fork 模型扩展到 select 模型时需要同时处理键盘输入和 socket 接收

1. 问题描述

第一次实验中，全双工聊天通过 `fork()` 实现。父进程负责接收对端消息，子进程负责读取键盘输入并发送消息。这种方式逻辑直观，但进程退出和资源回收相对复杂。

本次实验为了完成扩展要求，新增了基于 `select()` 的单进程全双工通信模型。在实现过程中遇到的主要问题是：程序既要监听用户键盘输入，又要监听 `socket` 上是否有来自对端的密文消息。如果只阻塞在 `std::getline()` 上，程序就无法及时接收网络消息；如果只阻塞在 `recvPacket()` 上，程序又无法及时响应用户输入。

2. 解决方法

解决方法是将标准输入 `STDIN_FILENO` 和通信 `socket` 同时加入 `fd_set`，由 `select()` 统一监听。每一轮循环中，程序先调用 `select()` 阻塞等待，当某个文件描述符可读时，再根据就绪对象执行对应逻辑：

1. 如果 `socket` 可读，说明对端发来了密文包，程序调用 `recvPacket()` 接收，再调用 `decryptMessage()` 解密显示。
2. 如果标准输入可读，说明用户输入了新消息，程序读取明文后调用 `encryptMessage()` 加密，再调用 `sendPacket()` 发送。
3. 如果收到或输入 `quit`，则跳出循环并关闭连接。

关键代码如下：

```
1 while (true)
2 {
3     // 1. 将标准输入和 socket 同时加入监听集合。
4     fd_set readfds;
5     FD_ZERO(&readfds);
6     FD_SET(STDIN_FILENO, &readfds);
7     FD_SET(sockfd, &readfds);
8
9     const int maxFd = std::max(STDIN_FILENO, sockfd);
10
11     // 2. 等待任意一个描述符变为可读。
12     int ret = ::select(maxFd + 1, &readfds, nullptr, nullptr,
13                         nullptr);
14     if (ret < 0)
15     {
16         if (errno == EINTR)
17         {
18             continue;
19         }
20         throw std::runtime_error("select() failed");
21     }
22
23     // 3. socket 可读：接收密文并解密显示。
24     if (FD_ISSET(sockfd, &readfds))
25     {
26         std::vector<unsigned char> cipherPacket;
27         if (!recvPacket(sockfd, cipherPacket))
28         {
29             break;
30         }
31
32         std::string plainText =
33             decryptMessage(cipherPacket, key, algorithmType);
34
35         std::cout << "Receive message form <" << peerName << ">:"
36             << plainText << "\n";
37     }
```

```
38         if (isQuitCommand(plainText))
39         {
40             break;
41         }
42     }
43
44     // 4. 标准输入可读：读取明文、加密并发送。
45     if (FD_ISSET(STDIN_FILENO, &readfds))
46     {
47         std::string inputLine;
48         if (!std::getline(std::cin, inputLine))
49         {
50             break;
51         }
52
53         std::vector<unsigned char> cipherPacket =
54             encryptMessage(inputLine, key, algorithmType);
55
56         if (!sendPacket(sockfd, cipherPacket))
57         {
58             break;
59         }
60
61         if (isQuitCommand(inputLine))
62         {
63             break;
64         }
65     }
66 }
```

Listing 25 select 同时监听标准输入和 socket

通过引入 `select()`，程序在单进程内实现了发送和接收的并发响应，不需要额外创建子进程，也降低了退出处理复杂度。测试结果表明，`select()` 模型下客户端和服务端均能够正常完成 RSA-DES 密钥交换、全双工聊天和 `quit` 退出。

七、实验总结与个人思考

通过本次实验，我在第一次实验“基于 DES 加密的 TCP 聊天程序”的基础上，进一步实现了基于 RSA 算法的 DES 会话密钥自动分配机制。程序整体上采用 RSA-DES 混合加密思路：RSA 负责解决对称密钥在网络中的安全分发问题，DES 负责后续聊天正文的快速加密与解密。相比第一次实验中通信双方需要手工输入相同密钥的方式，本次实验使客户端能够自动生成 DES 会话密钥，并通过服务端 RSA 公钥进行加密传输，服务端再使用私钥解密得到同一会话密钥，从而实现了更加完整的安全通信流程。

从实现过程来看，本实验并不是简单地分别实现 RSA 和 DES 两个算法，而是需要将密码算法、Socket 通信、应用层封包协议和全双工聊天逻辑结合起来。RSA 模块需要保证密钥生成、快速模幂、模逆元和分组加解密的正确性；密钥交换模块需要保证公钥包和加密密钥包能够被可靠传输和正确解析；聊天模块则需要在密钥交换完成后复用原有 DES 加密通信能力。通过这些模块之间的衔接，我更加深刻地理解了实际安全通信系统中非对称加密用于密钥协商，对称加密用于数据传输的基本设计思想。

本次实验中我体会最深的一点是，网络安全程序的正确性不仅取决于算法本身是否正确，还取决于工程实现细节是否可靠。例如，RSA 加密 DES 密钥时不能直接将 8 字节密钥整体作为一个整数处理，而需要根据 RSA 模数大小进行合理分组；RSA 公钥和加密密钥也不能直接发送 C++ 结构体，而应该使用明确的数据包格式和长度头封包机制；TCP 通信中还必须考虑粘包、拆包和完整收发问题。这些问题说明，密码算法从理论走向实际程序时，需要额外关注数据格式、边界条件、异常处理和跨平台表示等工程因素。

此外，本实验在保留 `fork()` 全双工通信模型的基础上，进一步实现了基于 `select()` 的单进程全双工通信。通过对比可以发现，`fork()` 模型思路直观，发送和接收天然分离；而 `select()` 模型能够在一个进程中同时监听标准输入和 socket，使通信逻辑更加集中，也减少了进程创建和回收的开销。这让我对 Linux I/O 多路复用机制有了更加直观的认识，也理解了为什么在更复杂的网络程序中，事件驱动模型往往比简单的多进程模型更易扩展。

总的来说，本次实验完成了从“手工共享 DES 密钥的加密聊天”到“RSA 自动分配 DES 会话密钥的加密聊天”的升级。通过本次实验，我不仅加深了对 RSA 和 DES 算法原理的理解，也进一步掌握了如何在 Linux 网络程序中设计可靠的安全通信流程。后续如果继续完善该系统，可以进一步引入更大的 RSA 密钥长度、使用更安全的分组加密模式如 CBC、CTR 或 GCM，并增加消息认证机制，从而使程序更加接近真实安全通信系统的设计要求。